

v3.0 · July 2026

# EMS Backend

## Technical Documentation

Employee Management System · REST API · Hostinger Production

|             |                                       |
|-------------|---------------------------------------|
| Version     | 3.0                                   |
| Repository  | Employee-management-system            |
| Runtime     | Node.js 20+ · Fastify 4 · Prisma 5    |
| Primary API | ems-api.saqibsaeed.cloud              |
| Database    | PostgreSQL (Hostinger VPS)            |
| Audience    | Backend engineers, architects, DevOps |

Client (blue)

API (green)

Database (orange)

External (purple)

# Table of Contents

---

- 0. Quick Start for New Developers
  - 0.1 Prerequisites
  - 0.2 First-Time Setup
  - 0.3 Verify Login (Swagger or curl)
  - 0.4 Where to Look First
- 1. Executive Summary
  - Key Characteristics
  - Production Endpoints
  - Scope of This Document
- 2. System Architecture Overview
  - 2.1 High-Level Architecture
  - 2.2 Request Lifecycle
  - 2.3 ASCII — Request Path Summary
  - 2.4 Registered Route Modules
- 2.5 Middleware Decision Tree — Which Middleware Runs When?
  - Tenant-Optional vs Protected Routes
- 3. Technology Stack
  - 3.1 Core Stack
  - 3.2 Removed / Deprecated
  - 3.3 Environment Variables (Names Only)
- 4. Project Structure
  - 4.1 Annotated Directory Tree
  - 4.2 Module Architecture Pattern
- 5. Multi-Tenancy Deep Dive
  - 5.1 Four-Layer Tenant Resolution Chain
  - 5.2 Resolution Priority Table
  - 5.3 Tenant-Optional Routes
  - 5.4 Login Auto-Resolution
  - 5.5 Seeded Tenants
- 6. Authentication & Authorization
  - 6.1 Auth Flow Overview
  - 6.2 JWT Access Token Payload
  - 6.3 Refresh Token Rotation
  - 6.4 MFA / OTP Policy
  - 6.5 Role Hierarchy
  - 6.6 Authorization Middleware
- 7. Module Reference
  - 7.1 Auth ( /auth/\* )

- 7.2 Employees ( /employees/\* )
- 7.3 Departments ( /departments/\* )
- 7.4 Attendance ( /attendance/\* )
- 7.5 Leave ( /leave/\* + /leave-engine/\* )
- 7.6 Timesheets ( /timesheets/\* )
- 7.7 Payroll ( /payroll/\* )
- 7.8 Holidays ( /holidays/\* )
- 7.9 Analytics ( /analytics/\* )
- 7.10 Reports & Export
- 7.11 Settings ( /settings/\* )
- 7.12 Notifications ( /notifications/\* )
- 7.13 Search ( /search )
- 7.14 Audit Logs ( /audit-logs/\* )
- 7.15 Dashboard
- 7.16 Recruitment ( /recruitment/\* )
- 7.17 Performance ( /performance/\* )
- 7.18 Assets ( /assets/\* )
- 7.19 Announcements ( /announcements/\* )
- 7.20 Billing ( /billing/\* )
- 7.21 Logs ( /logs )
- 7.22 Leave Engine ( /leave-engine/\* ) — Policy Engine Detail
- 7.23 Payout ( /payroll/me/payout-methods )
- 7.24 Holidays Policy ( /holidays/policy/\* )
- 7.25 Timesheets Config ( /timesheets/config/\* )
- 7.26 Auth Submodule Map
- 7.27 Settings & Integrations ( /settings/\* )
- 7.28 Module → Prisma Model Quick Map

## 8. Database Schema Overview

- 8.1 Model Inventory (78 Prisma Models)
- 8.1b Domain Grouping (Summary Table)
- 8.2 Core Entity Relationships
- 8.3 Multi-Tenant Isolation

## 9. Payroll Engine

- 9.1 Overview
- 9.2 Supported Countries (Built-in Metadata)
- 9.3 Payroll Run Flow
- 9.4 Statutory Pack Resolution
- 9.5 Component Calculation Types
- 9.6 Payslip Generation

## 10. External Integrations

- 10.1 Resend (Email)
- 10.2 Cloudinary (File Upload)
- 10.3 Notifications SSE Stream
- 10.4 Render (Legacy Deployment)
- 10.5 Hostinger (Primary Production)
- 11. Error Handling & Logging
  - 11.1 Response Envelope
  - 11.2 Error Handler Pipeline
  - 11.3 HTTP Status Contract
  - 11.4 Structured Logging
- 12. Testing Strategy
  - 12.1 Test Pyramid
  - 12.2 Test Categories
  - 12.3 Test Database Safety
  - 12.4 CI Pipeline
- 13. Deployment
  - 13.1 Hostinger VPS Architecture
  - 13.2 ASCII — Hostinger Directory Layout
  - 13.3 CI/CD Pipeline (GitHub Actions → Hostinger)
  - 13.6 Hostinger Deep Dive (Production)
  - 13.7 Render vs Hostinger Comparison
  - 13.4 Render (Legacy)
  - 13.5 Local Development
- 14. Security Considerations
  - 14.1 Authentication Security
  - 14.2 Multi-Tenant Security
  - 14.3 Input Validation
  - 14.4 HTTP Security Headers
  - 14.5 Data Protection
- 15. Known Issues (E2E Audits)
  - Summary (28 documented issues)
  - Recommendations
- 16. API Reference Overview
  - 16.1 Interactive Documentation
  - 16.2 Response Envelope (All Endpoints)
  - 16.3 Quick Test
  - 16.4 Auth Headers After Login
- 17. End-to-End Workflow Diagrams
  - 17.1 Payroll Engine — Gross → Deductions → Statutory → Net
  - 17.2 India Statutory — PF / ESI / PT / TDS
  - 17.3 KWD Work Week + Currency Flow

- 17.4 Leave Balance Accrual + Approval
- 17.5 Attendance Check-In/Out + Regularization
- 17.6 Timesheet Lock / Unlock / Submit
- 17.7 Employee Soft Delete Cascade
- 17.8 Department Hierarchy Tree Resolution
- 17.9 Export Job Lifecycle (ExcelJS)
- 17.10 Email Flow — Resend (Password Reset + OTP)
- 17.11 Session Family ID Reuse Detection
- 17.12 Rate Limiting Flow
- 17.13 CORS + Helmet Middleware Chain
- 17.14 Swagger Registration Order
- 17.15 Prisma Transaction Patterns
- 17.16 Audit Log Write on Mutation
- 17.17 Search Multi-Entity Flow
- 17.18 Report Scheduling (CRUD — cron execution)

18. Utilities Reference ( `src/utls/` )

19. Middleware & Plugins Reference

Middleware ( `src/middleware/` )

Plugins ( `src/plugins/` )

Jobs ( `src/jobs/` + `src/lib/` )

Appendix A — Glossary (Expanded)

Appendix B — Document Revision History

Appendix C — Complete API Route Index

Auth & Identity

Employees & Departments

Attendance & Leave

Timesheets & Payroll

Platform & Admin

Health & Docs (outside prefix)

Appendix D — Developer Troubleshooting

Useful Commands

## 0. Quick Start for New Developers

**In simple terms:** Clone the repo, start local Postgres, migrate, seed, and hit Swagger — you will have a working API in about 15 minutes.

**What it does:** Gets a new engineer from zero to a working local API in under 15 minutes.

**Why it matters:** Onboarding speed reduces mistakes against production data.

**How it works:** Clone → local Postgres → migrate → seed → dev server → Swagger login.

### 0.1 Prerequisites

| Tool    | Version    | Purpose                 |
|---------|------------|-------------------------|
| Node.js | ≥ 20       | Runtime (ES modules)    |
| Docker  | any recent | Local PostgreSQL only   |
| Git     | 2.x        | Clone + branch workflow |

### 0.2 First-Time Setup

1. Clone the repository and install dependencies.
2. Start PostgreSQL via Docker Compose.
3. Point `DATABASE_URL` at the local database.
4. Run Prisma migrations and seed data.
5. Start the dev server and open Swagger.

```
git clone https://github.com/saeedafri/Employee-management-system.git EMS
cd EMS
npm ci
docker compose up -d
export DATABASE_URL=postgresql://ems:ems_local_dev@127.0.0.1:5432/ems_dev
npx prisma migrate deploy
npm run db:seed
npm run dev
```

### 0.3 Verify Login (Swagger or curl)

```
curl -s -X POST http://localhost:3000/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -H "X-Tenant-Key: acme-corp-001" \
  -d '{"email":"hr@acme.test","password":"Password123!"}' | jq .
```

Open **http://localhost:3000/docs** — authorize with the returned `accessToken` .

0.4 Where to Look First

| Task                | Start here                                                                                               |
|---------------------|----------------------------------------------------------------------------------------------------------|
| Add an API endpoint | <code>src/modules/&lt;domain&gt;/*.routes.js</code> → controller → service → repository                  |
| Change auth rules   | <code>src/middleware/authenticate.js</code>                                                              |
| Tenant scoping      | <code>src/middleware/resolveTenant.js</code> + always <code>tenantId</code> in Prisma <code>where</code> |
| Payroll math        | <code>src/modules/payroll/</code> + <code>src/utils/statutoryCalculation.js</code>                       |
| API contract        | <code>docs/API_MAPPING.md</code> + <code>src/plugins/swagger.js</code>                                   |
| Deploy              | <code>.github/workflows/deploy-hostinger.yml</code>                                                      |

1. Executive Summary

**In simple terms:** EMS is a multi-tenant HR API — one codebase serves many companies, each isolated by tenant.  
*This section orients architects and new backend engineers on scope, endpoints, and design goals.*

The **EMS (Employee Management System) Backend** is a production-grade, multi-tenant Human Resource Management System (HRMS) REST API. It powers employee lifecycle management, attendance, leave, timesheets, payroll, analytics, and administrative workflows for organizations operating across multiple countries.

## Key Characteristics

| Dimension      | Description                                                                                         |
|----------------|-----------------------------------------------------------------------------------------------------|
| Architecture   | Modular monolith — Fastify v4 with layered route → controller → service → repository pattern        |
| Data isolation | Shared PostgreSQL database with row-level <code>tenantId</code> scoping on every table              |
| Authentication | JWT access tokens (15 min) + opaque refresh token rotation (httpOnly cookie, 30 days)               |
| Authorization  | Role-based via <code>memberType</code> enum; <code>SUPER_ADMIN</code> bypasses all role checks      |
| Deployment     | Primary: Hostinger VPS (Docker + nginx + PM2). Legacy: Render Web Service                           |
| API contract   | OpenAPI/Swagger at <code>/docs</code> ; canonical field mapping in <code>docs/API_MAPPING.md</code> |

## Production Endpoints

```
Primary (Hostinger): https://ems-api.saqibsaeed.cloud/api/v1
Legacy (Render):     https://employee-management-system-2b9q.onrender.com/api/v1
Health:              GET /health → { "status": "ok" }
Swagger:             GET /docs
```

## Scope of This Document

This document is intended for **backend developers**, **solution architects**, and **DevOps engineers** integrating with or operating the EMS API. It covers request lifecycles, tenant resolution, auth flows, module boundaries, the payroll calculation engine, deployment topology, and operational concerns.

---

---

---

---

---

---

---

---

---

---

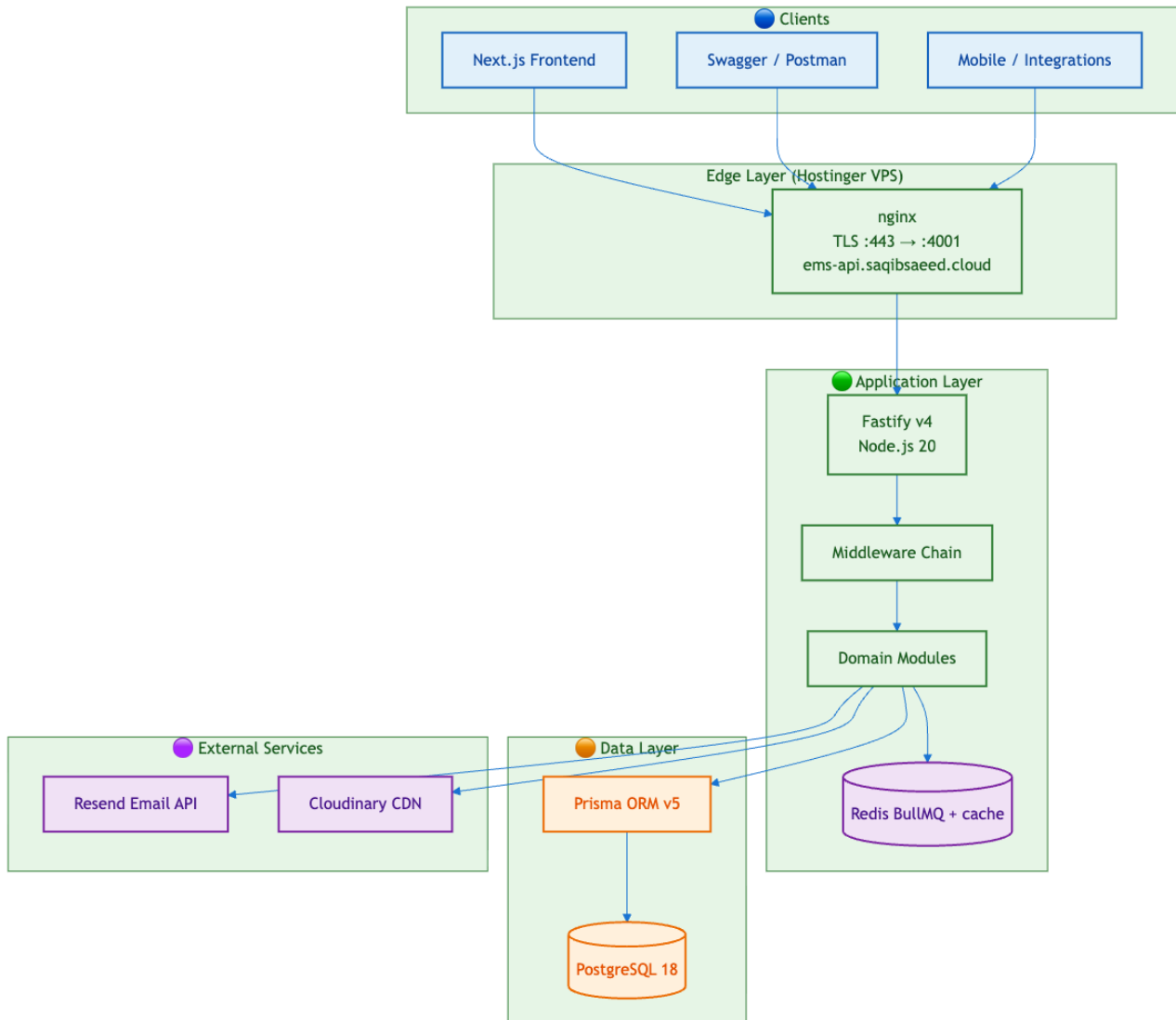


## 2. System Architecture Overview

**In simple terms:** Every request flows through nginx → Fastify middleware → domain modules → Prisma → PostgreSQL.

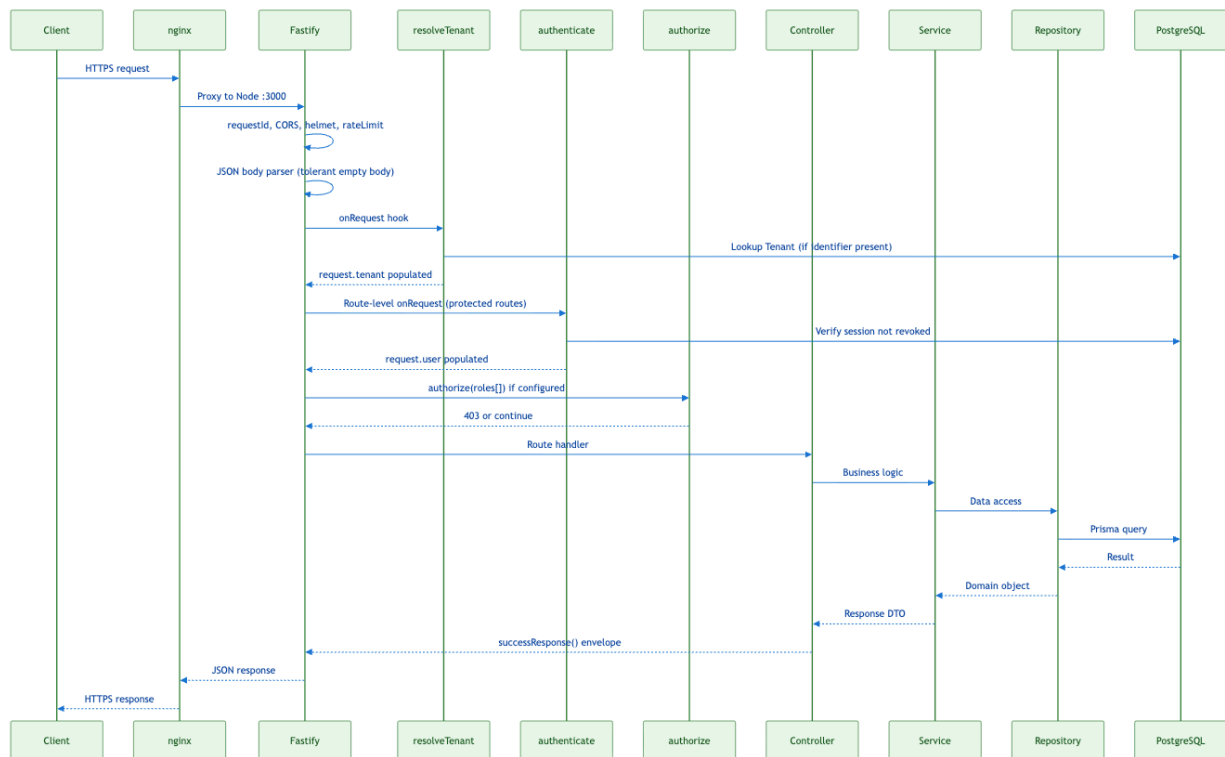
*Understand the layers before diving into individual modules or auth rules.*

### 2.1 High-Level Architecture



### 2.2 Request Lifecycle

Every HTTP request passes through a deterministic pipeline before reaching domain logic.



## 2.3 ASCII — Request Path Summary

```

Client
|
▼
nginx (TLS, X-Forwarded-*)
|
▼
Fastify plugins: requestId → cookie → prisma → cors → helmet → rateLimit → multipart
|
▼
Global: requestLogging hook, errorHandler
|
▼
/api/v1/* prefix group:
  onRequest: resolveTenant
  Route: onRequest: [authenticate, authorize(...)]
  Handler: controller → service → repository → Prisma
|
▼
PostgreSQL (tenant-scoped rows)
  
```

## 2.4 Registered Route Modules

All domain routes register under `config.apiPrefix ( /api/v1 )` in `src/app.js` :

| Module | Route Prefix | Primary Concern                     |
|--------|--------------|-------------------------------------|
| Auth   | /auth/*      | Login, refresh, MFA, password reset |

| Module               | Route Prefix               | Primary Concern               |
|----------------------|----------------------------|-------------------------------|
| Employees            | /employees/*               | CRUD, documents, photo upload |
| Departments          | /departments/*             | Org hierarchy                 |
| Holidays             | /holidays/*                | Calendar management           |
| Leave                | /leave/*                   | Requests, balances, approvals |
| Leave Engine         | /leave-engine/*            | Policy engine, ledger         |
| Attendance           | /attendance/*              | Check-in/out, regularization  |
| Timesheets           | /timesheets/*              | Time entries, projects        |
| Timesheets Config    | /timesheets/config/*       | Templates, settings           |
| Payroll              | /payroll/*                 | Runs, payslips, statutory     |
| Payout               | /payroll/me/payout-methods | Employee bank details         |
| Analytics            | /analytics/*               | HR dashboards                 |
| Reports              | /reports/*                 | Scheduled reports             |
| Export               | /export/*                  | Async CSV/Excel exports       |
| Settings             | /settings/*                | Tenant config, integrations   |
| Notifications        | /notifications/*           | In-app + SSE stream           |
| Search               | /search                    | Global search                 |
| Audit Logs           | /audit-logs/*              | Immutable audit trail         |
| Dashboard (Manager)  | /manager/*                 | Manager views                 |
| Dashboard (Employee) | /employee/*                | Employee self-service         |
| Recruitment          | /recruitment/*             | Job openings, candidates      |
| Performance          | /performance/*             | Reviews, goals                |
| Assets               | /assets/*                  | Asset tracking                |
| Announcements        | /announcements/*           | Company comms                 |
| Billing              | /billing/*                 | Subscription billing          |
| Logs                 | /logs                      | Application log entries       |

---

---

---

---

---

---

---

---

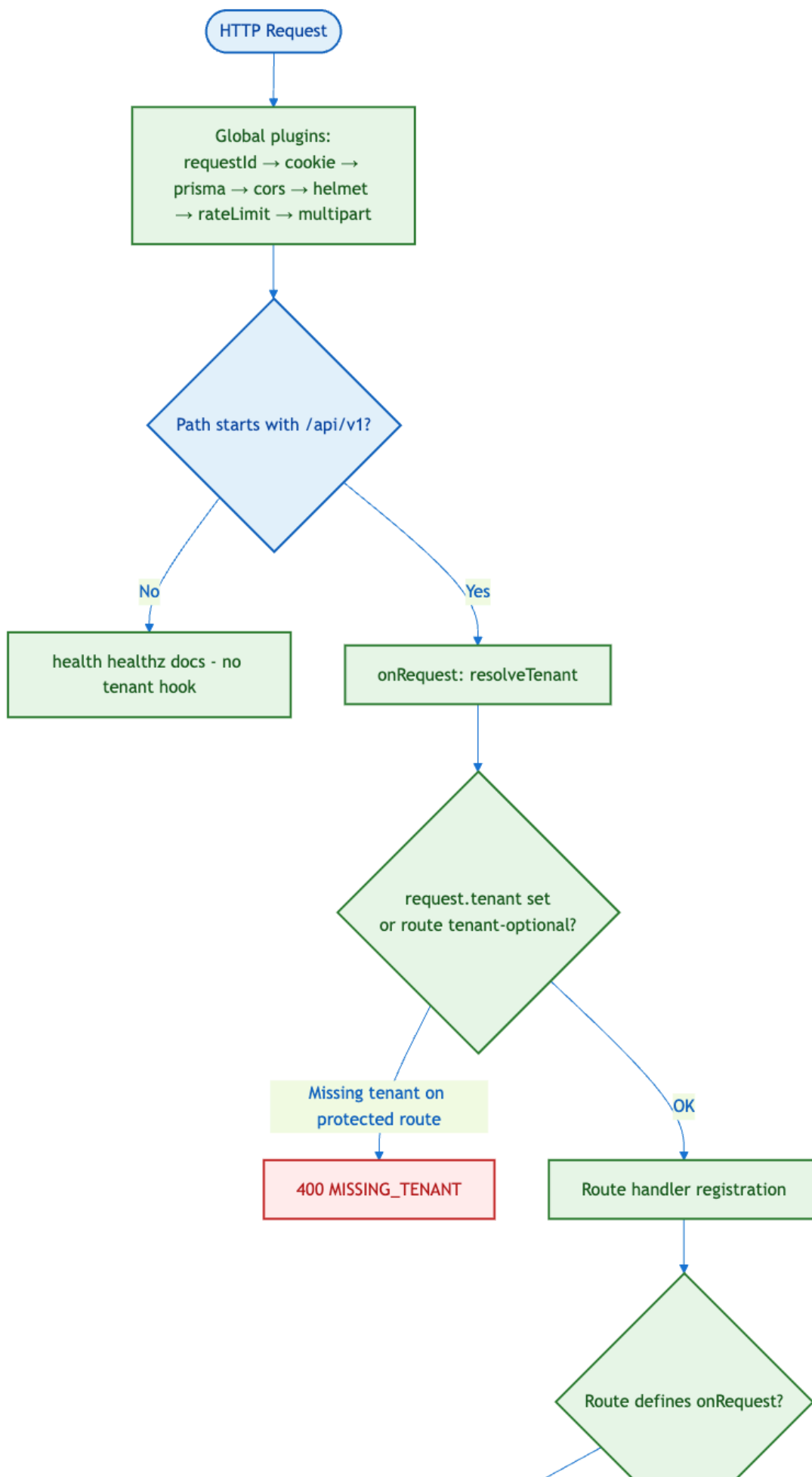
---

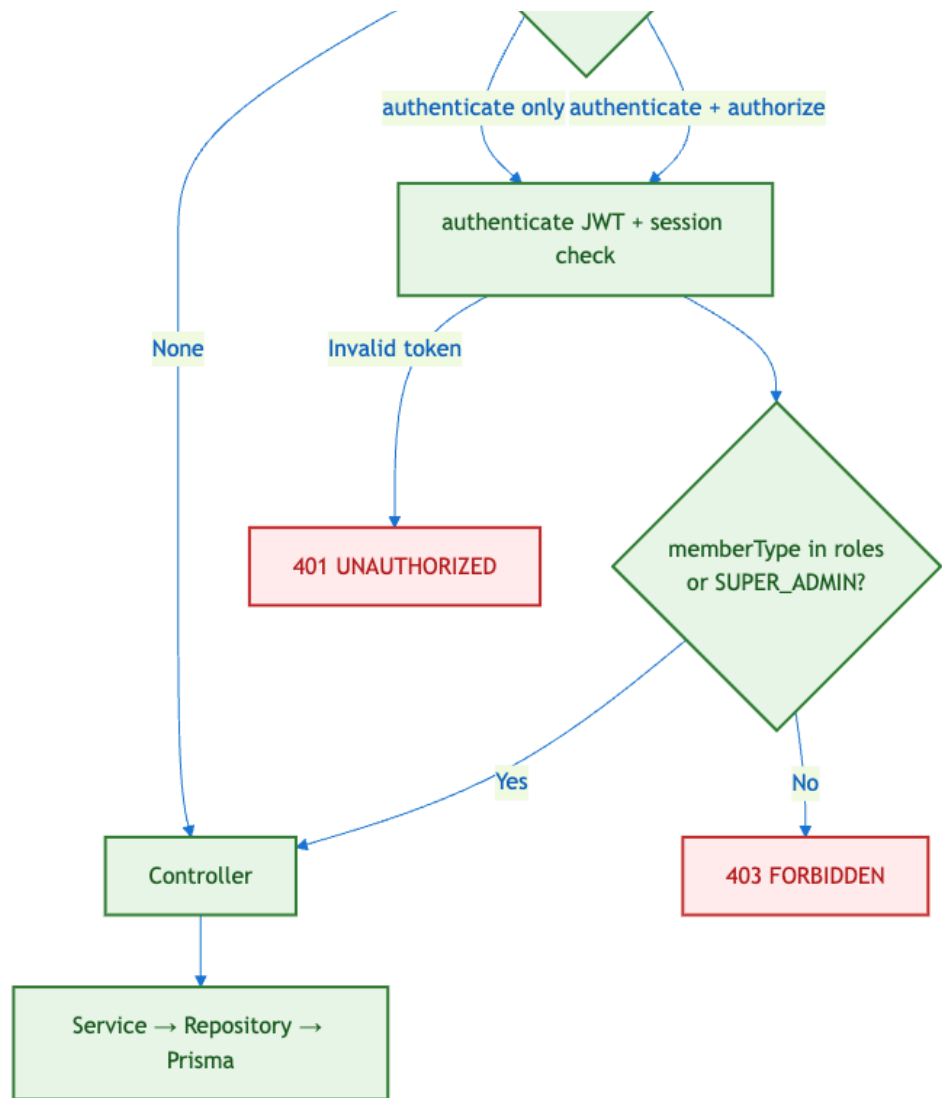
## 2.5 Middleware Decision Tree — Which Middleware Runs When?

**What it does:** Shows exactly which hooks fire for public vs protected routes.

**Why it matters:** Misunderstanding middleware order causes auth bugs and missing tenant context.

**How it works:** Follow the tree from HTTP method + path to the final handler.





## Tenant-Optional vs Protected Routes

| Route pattern          | resolveTenant             | authenticate | authorize             |
|------------------------|---------------------------|--------------|-----------------------|
| POST /auth/login       | optional (email resolves) | —            | —                     |
| GET /health            | — (outside prefix)        | —            | —                     |
| GET /employees         | required                  | yes          | —                     |
| POST /employees        | required                  | yes          | HR_ADMIN, SUPER_ADMIN |
| GET /analytics/summary | required                  | yes          | HR_ADMIN, SUPER_ADMIN |

---

---

---

---

---

## 3. Technology Stack

**In simple terms:** Node 20, Fastify 4, Prisma 5, and PostgreSQL form the core; integrations handle email, files, and exports.

*Versions and libraries listed here are the supported production stack.*

### 3.1 Core Stack

| Layer            | Technology       | Version / Notes                                              |
|------------------|------------------|--------------------------------------------------------------|
| Runtime          | Node.js          | ≥ 20.0.0, ES Modules ( "type": "module" )                    |
| Framework        | Fastify          | v4 — plugins for CORS, helmet, rate-limit, cookie, multipart |
| ORM              | Prisma           | v5 — no raw SQL; always Prisma Client                        |
| Database         | PostgreSQL       | v18 (local Docker + Hostinger production)                    |
| Auth tokens      | jose             | JWT access tokens                                            |
| Password hashing | Argon2id         | type:2, memoryCost:19456, timeCost:2                         |
| Validation       | Zod              | Request body validation in services                          |
| Logging          | Pino             | Structured JSON logs; pino-pretty in dev                     |
| API docs         | @fastify/swagger | OpenAPI spec at /docs                                        |
| Image processing | sharp            | WebP conversion before Cloudinary upload                     |

| Layer        | Technology                    | Version / Notes                          |
|--------------|-------------------------------|------------------------------------------|
| Email        | Resend HTTP API               | Primary provider for OTP/password reset  |
| File storage | Cloudinary                    | Documents, employee photos, tenant logos |
| Export       | ExcelJS                       | Styled XLSX exports                      |
| Testing      | Node test runner + Playwright | Unit, integration, E2E scripts           |

### 3.2 Removed / Deprecated

| Component                     | Status                                      |
|-------------------------------|---------------------------------------------|
| Redis / BullMQ                | <b>Removed</b> — all operations synchronous |
| MySQL                         | <b>Replaced</b> by PostgreSQL (2026-06)     |
| Queue-based email/export jobs | <b>Removed</b> — inline processing          |

### 3.3 Environment Variables (Names Only)

| Category       | Variable Names                                                                                                                                         |
|----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| Core           | NODE_ENV , PORT , LOG_LEVEL , APP_NAME , APP_VERSION , API_PREFIX , API_URL                                                                            |
| Database       | DATABASE_URL                                                                                                                                           |
| JWT / Session  | JWT_SECRET , ACCESS_TOKEN_EXPIRES_IN , REFRESH_TOKEN_EXPIRES_IN , SESSION_COOKIE_NAME , SESSION_MAX_AGE_DAYS                                           |
| Multi-tenant   | APP_DOMAIN , DEFAULT_TENANT_KEY                                                                                                                        |
| CORS           | CORS_ORIGIN                                                                                                                                            |
| Email          | EMAIL_PROVIDER , SMTP_HOST , SMTP_PORT , SMTP_USER , SMTP_PASS , SMTP_FROM , RESEND_API_KEY , RESEND_FROM , BREVO_API_KEY , BREVO_FROM , SUPPORT_EMAIL |
| Password reset | RESET_PASSWORD_TOKEN_TTL_MINUTES , RESET_PASSWORD_RATE_LIMIT_MAX , RESET_PASSWORD_RATE_LIMIT_WINDOW , FRONTEND_RESET_PASSWORD_URL                      |
| Invitations    | INVITE_TOKEN_TTL_HOURS , FRONTEND_APP_URL                                                                                                              |
| File storage   | CLOUDINARY_CLOUD_NAME , CLOUDINARY_API_KEY , CLOUDINARY_API_SECRET                                                                                     |
| Exports        | EXPORTS_DIR                                                                                                                                            |



| Category | Variable Names                    |
|----------|-----------------------------------|
| Legacy   | REDIS_URL (ignored, returns null) |

**Production guard:** `JWT_SECRET` must be set to a non-default value when `NODE_ENV=production` — the app fails closed on startup otherwise.

## 4. Project Structure

**In simple terms:** Code is organized by domain module, each following routes → controller → service → repository.

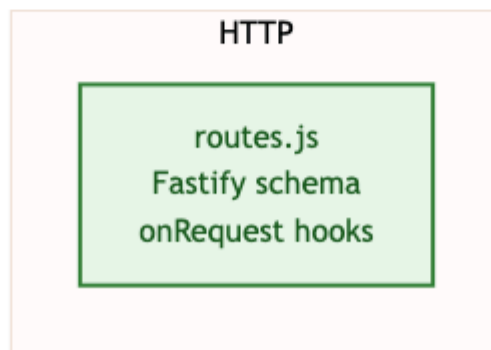
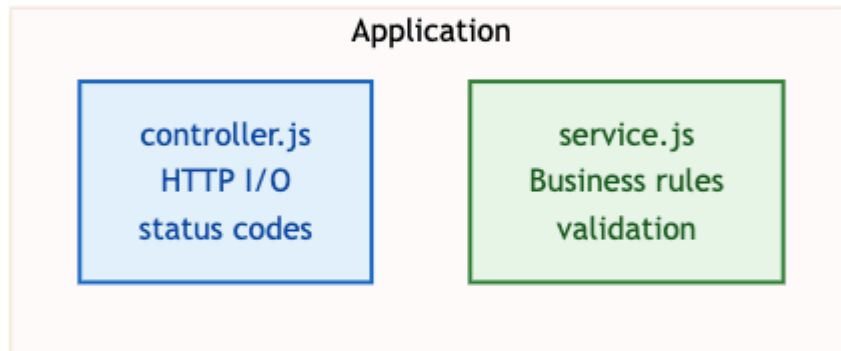
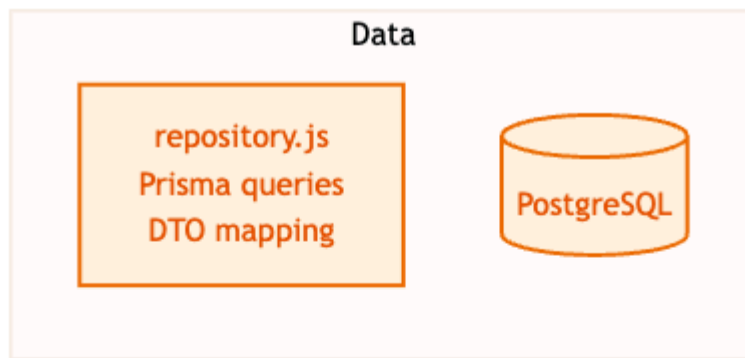
*Use this map to find where to add endpoints or trace a bug through the stack.*

## 4.1 Annotated Directory Tree

```
EMS/
├── src/
│   ├── app.js                # Fastify factory – plugin + route registration
│   ├── server.js             # Entry point (createApp → listen)
│   ├── config/
│   │   └── index.js          # Centralized env-based configuration
│   ├── middleware/
│   │   ├── authenticate.js   # JWT verify + authorize(roles[])
│   │   ├── resolveTenant.js   # 4-layer tenant resolution
│   │   ├── errorHandler.js   # Global Fastify error handler
│   │   └── requestLogging.js  # Per-request Pino logging hook
│   ├── plugins/
│   │   ├── prisma.js         # Prisma client decorator
│   │   ├── cors.js, helmet.js, rateLimit.js, requestId.js
│   │   └── swagger.js        # OpenAPI spec (registered AFTER routes)
│   ├── modules/              # Domain modules (see §7)
│   │   └── <module>/
│   │       ├── <module>.routes.js
│   │       ├── <module>.controller.js
│   │       ├── <module>.service.js
│   │       ├── <module>.repository.js
│   │       ├── <module>.validator.js (where applicable)
│   │       └── <module>.policy.js (where applicable)
│   └── utils/
│       ├── hash.js, token.js, response.js, id.js, logger.js
│       ├── clouinary.js, money.js, pagination.js
│       ├── statutoryCalculation.js # Payroll statutory engine
│       └── sseClients.js         # SSE notification clients
├── prisma/
│   ├── schema.prisma         # 70+ models (see §8)
│   ├── migrations/           # Versioned SQL migrations
│   └── seed*.js               # Idempotent seed scripts
├── tests/                     # Unit, integration, E2E
├── scripts/                   # Deployed UI audit scripts (phase4)
├── docs/
│   ├── API_MAPPING.md        # Canonical API response shapes
│   └── E2E_BACKEND_ISSUES.md # Known issues from audits
├── .github/workflows/
│   ├── ci.yml                # Lint + build check
│   └── deploy-hostinger.yml   # Auto-deploy to VPS
└── docker-compose.yml         # Local Postgres only
```

## 4.2 Module Architecture Pattern

Every domain module follows a consistent layered architecture.



#### Responsibilities:

| Layer             | Responsibility                                      | Must NOT               |
|-------------------|-----------------------------------------------------|------------------------|
| <b>routes</b>     | URL mapping, OpenAPI schema, middleware chain       | Contain business logic |
| <b>controller</b> | Parse request, call service, format response        | Direct Prisma calls    |
| <b>service</b>    | Business rules, authorization checks, orchestration | Know about HTTP        |
| <b>repository</b> | Prisma queries, data mapping                        | Business decisions     |

---

---

---

---

---

---

---

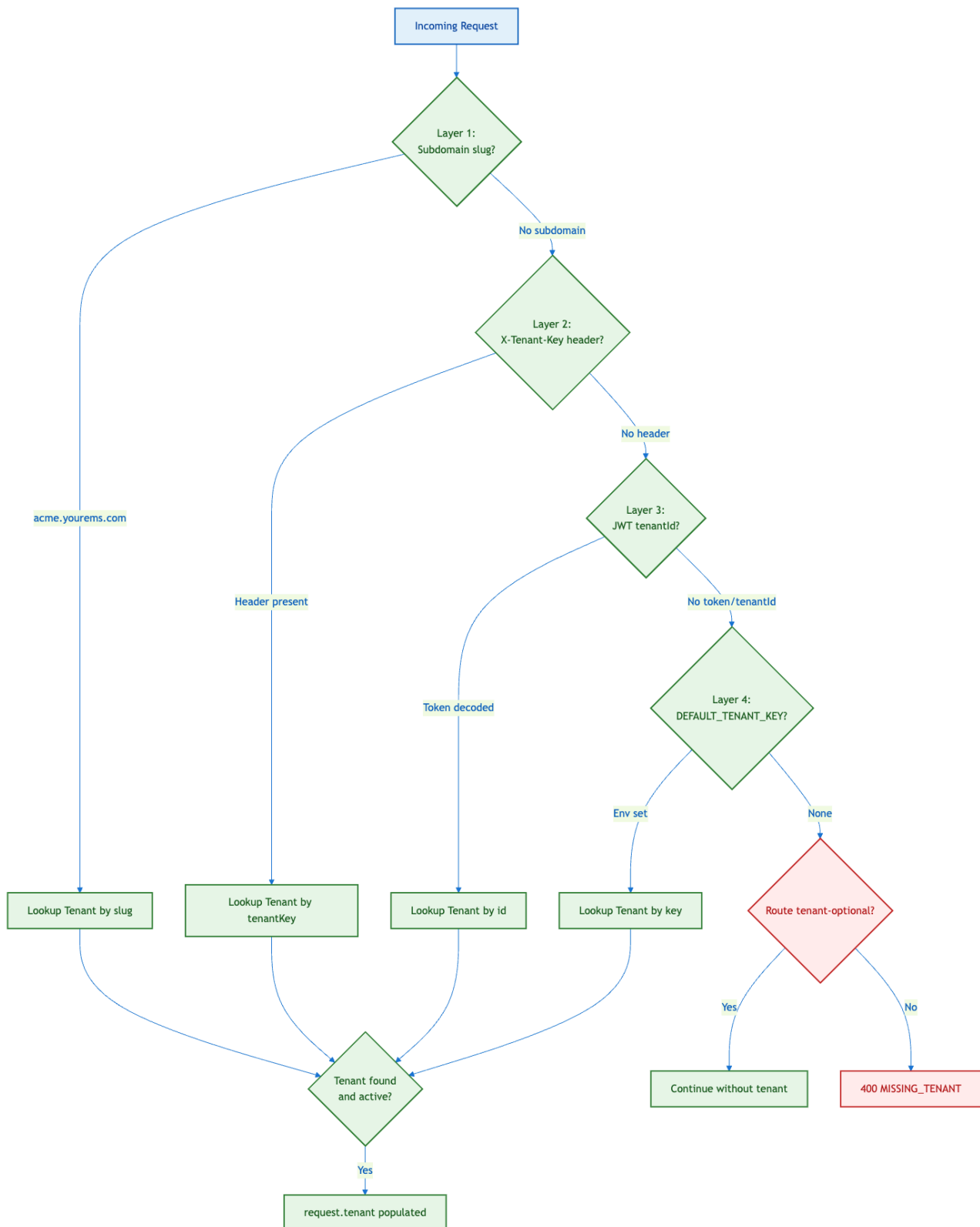
## 5. Multi-Tenancy Deep Dive

**In simple terms:** Every database row belongs to exactly one tenant — isolation is enforced at query time.

*Tenant resolution runs before protected handlers; never query without `tenantId` .*

EMS uses a **shared-database, shared-schema** multi-tenant model. Every table includes a `tenantId` foreign key. Data isolation is enforced at the application layer via Prisma `where` clauses.

## 5.1 Four-Layer Tenant Resolution Chain



## 5.2 Resolution Priority Table

| Priority | Source                | Example                      | Lookup Field |
|----------|-----------------------|------------------------------|--------------|
| 1        | Host header subdomain | acme.yourems.com → slug acme | Tenant.slug  |

| Priority | Source                 | Example                | Lookup Field     |
|----------|------------------------|------------------------|------------------|
| 2        | X-Tenant-Key header    | acme-corp-001          | Tenant.tenantKey |
| 3        | JWT payload tenantId   | After login, automatic | Tenant.id        |
| 4        | DEFAULT_TENANT_KEY env | Dev/testing fallback   | Tenant.tenantKey |

### 5.3 Tenant-Optional Routes

These routes resolve tenant internally (typically from email in request body):

- POST /auth/login , /auth/admin/login , /auth/refresh
- POST /auth/forgot-password , /auth/reset-password , /auth/validate-reset-token
- POST /auth/verify-otp , /auth/resend-otp , /auth/otp/initiate
- POST /auth/register

### 5.4 Login Auto-Resolution

When a user logs in with email + password (no tenant header):

1. Email is unique across tenants → tenant resolved automatically
2. Email exists in multiple tenants → AMBIGUOUS\_EMAIL error; client must send X-Tenant-Key
3. After login, JWT carries tenantId — all subsequent requests are tenant-scoped automatically

### 5.5 Seeded Tenants

| tenantKey          | Name             | Country | Currency | Purpose                            |
|--------------------|------------------|---------|----------|------------------------------------|
| acme-corp-001      | Acme Corp        | IN      | INR      | Primary test tenant (79 employees) |
| test-key-123456789 | Test Org         | —       | —        | Secondary test tenant              |
| kwd-litmus-001     | Kuwait Litmus Co | KW      | KWD      | Multi-country / locale testing     |

---

---

---

---

---

---

---

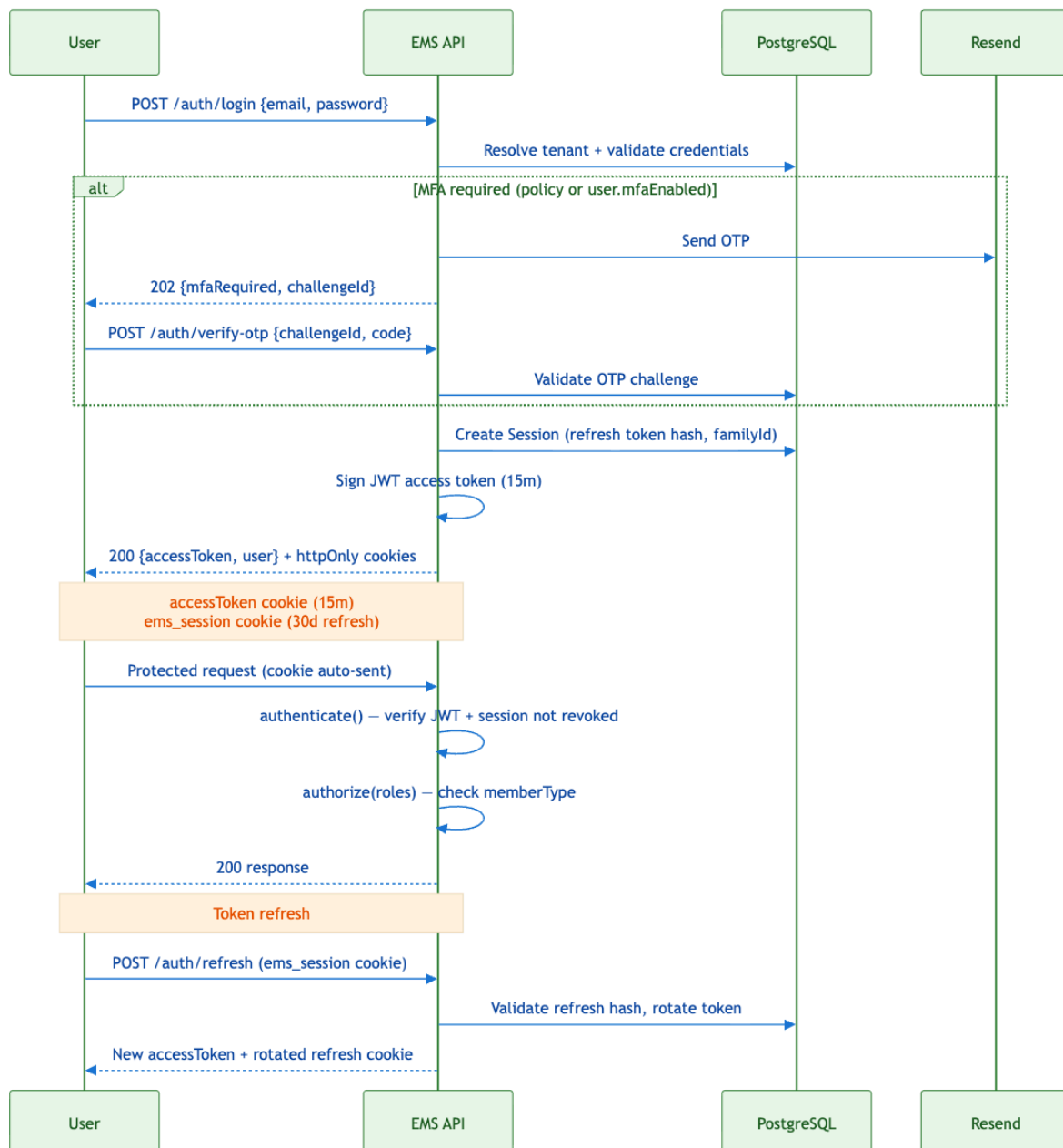
---

## 6. Authentication & Authorization

**In simple terms:** Short-lived JWT access tokens plus rotating httpOnly refresh cookies secure sessions.

*Roles ( memberType ) gate endpoints; SUPER\_ADMIN bypasses all role checks.*

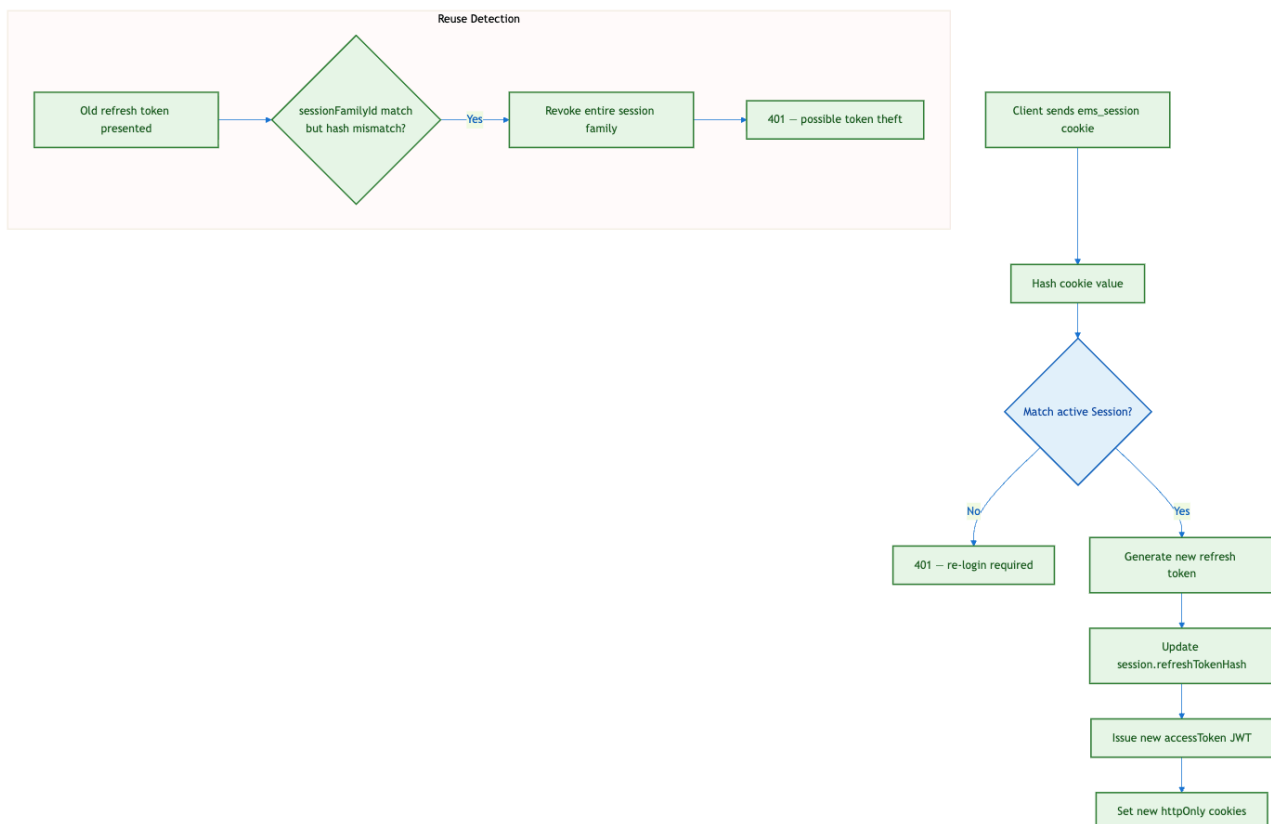
### 6.1 Auth Flow Overview



## 6.2 JWT Access Token Payload

| Claim       | Description                                                |
|-------------|------------------------------------------------------------|
| sub         | User ID                                                    |
| tenantId    | Tenant ID (for row scoping)                                |
| sessionId   | Session ID (revocation check)                              |
| memberType  | Role enum (SUPER_ADMIN, HR_ADMIN, etc.)                    |
| employeeId  | Linked employee ID (nullable)                              |
| permissions | Fine-grained permission keys (from Role/Permission tables) |

## 6.3 Refresh Token Rotation



## 6.4 MFA / OTP Policy

MFA is **policy-driven**, not globally enforced:

| ems_policy Value | Who Must MFA           |
|------------------|------------------------|
| REQUIRED_ALL     | Every user             |
| REQUIRED_ADMINS  | SUPER_ADMIN + HR_ADMIN |



| <code>mfa_policy</code> Value | Who Must MFA                                        |
|-------------------------------|-----------------------------------------------------|
| OPTIONAL                      | Only users with <code>user.mfaEnabled = true</code> |

OTP delivery via Resend email. Challenge stored in `otpChallenge` table with rate limiting.

### 6.5 Role Hierarchy

SUPER\_ADMIN  
|  
HR\_ADMIN  
|  
MANAGER  
|  
EMPLOYEE  
|  
AUDITOR

→ bypasses ALL `authorize()` checks  
→ full HR operations, settings, analytics  
→ team visibility, leave/attendance approvals  
→ self-service only (own data)  
→ read-only (same routes as EMPLOYEE currently)

| Role        | Typical Access                                                        |
|-------------|-----------------------------------------------------------------------|
| SUPER_ADMIN | All endpoints; may lack employee record for <code>/me/*</code> routes |
| HR_ADMIN    | Employee CRUD, payroll runs, settings, exports                        |
| MANAGER     | Team attendance/leave, manager dashboard                              |
| EMPLOYEE    | Own attendance, leave, payslips, documents                            |
| AUDITOR     | Enum exists; no dedicated route set yet                               |

### 6.6 Authorization Middleware

```
// authenticate.js – every protected route
onRequest: [authenticate, authorize(['HR_ADMIN', 'SUPER_ADMIN'])]

// SUPER_ADMIN hardcoded bypass in authorize():
if (memberType === 'SUPER_ADMIN') return;
```

**Note:** `Role` / `Permission` / `RolePermission` models exist in Prisma but `authorize()` uses `memberType` enum only. JWT carries `permissions[]` for future fine-grained checks.

## 7. Module Reference

**In simple terms:** Each domain (employees, leave, payroll, etc.) is a self-contained Fastify route group. Pick the module that matches your feature; follow the layered pattern inside it.

### 7.1 Auth ( /auth/\* )

| Endpoint Group | Key Operations                                                           |
|----------------|--------------------------------------------------------------------------|
| Login          | POST /login , POST /admin/login — tenant auto-resolve from email         |
| Session        | POST /refresh , POST /logout , POST /logout-all , GET /sessions          |
| Profile        | GET /me , PATCH /me/mfa                                                  |
| Password       | POST /forgot-password , POST /reset-password , GET /validate-reset-token |
| MFA            | POST /verify-otp , POST /resend-otp , POST /otp/initiate                 |
| Registration   | POST /register — creates tenant + first SUPER_ADMIN                      |

### 7.2 Employees ( /employees/\* )

- CRUD with soft delete ( deletedAt )
- POST /:id/documents — multipart upload → Cloudinary (WebP for images)
- POST /:id/photo — profile photo (sharp → WebP → Cloudinary)
- GET /employees/export/csv — HR\_ADMIN export
- Role isolation: EMPLOYEE sees own; HR sees all

### 7.3 Departments ( /departments/\* )

- Hierarchical tree ( parentId , depth )
- Optional department head ( headEmployeeId )
- Cycle detection on parent assignment

## 7.4 Attendance ( /attendance/\* )

| Endpoint                          | Description                               |
|-----------------------------------|-------------------------------------------|
| POST /check-in                    | Daily check-in with workMode, location    |
| POST /check-out                   | Check-out with duration calculation       |
| GET /records                      | Own records; ?month=YYYY-MM or date range |
| GET /team/records                 | MANAGER/HR_ADMIN team view                |
| GET /summary                      | Monthly summary stats                     |
| POST /regularization              | Request attendance correction             |
| PATCH /regularization/:id/approve | Manager approval                          |

## 7.5 Leave ( /leave/\* + /leave-engine/\* )

- Leave types, balances, requests with approval workflow
- Leave engine: policies, assignments, ledger transactions, comp-off
- Statuses: PENDING → APPROVED/DENIED/WITHDRAWN/CANCELLED

## 7.6 Timesheets ( /timesheets/\* )

- Projects, tasks, time entries
- Templates and tenant-level settings
- Integration with payroll (salary context for rate calculation)

## 7.7 Payroll ( /payroll/\* )

See [§9 Payroll Engine](#) for calculation details.

Key surfaces: pay groups, salary components, employee salaries, payroll runs, payslips, statutory packs, tax declarations, reimbursements, loans, garnishments.

## 7.8 Holidays ( /holidays/\* )

- Tenant holiday calendar with optional/mandatory flags
- Holiday policies and employee optional selections
- Used by attendance and payroll working-day calculations

## 7.9 Analytics ( /analytics/\* )

| Endpoint     | Roles                 |
|--------------|-----------------------|
| GET /summary | HR_ADMIN, SUPER_ADMIN |

| Endpoint                     | Roles                 |
|------------------------------|-----------------------|
| GET /attendance              | HR_ADMIN, SUPER_ADMIN |
| GET /headcount-by-department | HR_ADMIN, SUPER_ADMIN |
| GET /recent-activity         | HR_ADMIN, SUPER_ADMIN |
| GET /leave-summary           | HR_ADMIN, SUPER_ADMIN |

## 7.10 Reports & Export

- **Reports:** attendance, leave, payroll reports; scheduled report CRUD
- **Export:** async job queue (synchronous processing) — CSV/Excel/JSON for employees, attendance, leave

## 7.11 Settings ( /settings/\* )

- Tenant configuration, email templates, roles/permissions
- Security auth settings ( mfa\_policy )
- Integration endpoints (storage, email provider status)

## 7.12 Notifications ( /notifications/\* )

See [§10.3 Notifications SSE](#).

## 7.13 Search ( /search )

GET /search?q=<term> — searches employees, departments, leave, holidays in parallel.

## 7.14 Audit Logs ( /audit-logs/\* )

Immutable change tracking for entity mutations. DPIA report generation and CSV export.

## 7.15 Dashboard

| Prefix      | Audience | Key Endpoints                    |
|-------------|----------|----------------------------------|
| /manager/*  | MANAGER  | dashboard, team, approvals       |
| /employee/* | EMPLOYEE | dashboard, documents, team peers |

## 7.16 Recruitment ( /recruitment/\* )

**What it does:** Job openings and candidate pipeline for HR hiring workflows.

**Why it matters:** Bridges HRMS with talent acquisition without a separate ATS.

**How it works:** `recruitment.routes.js` → `service` → `Prisma JobOpening` + `Candidate` models.

| Endpoint area     | Roles    | Notes                             |
|-------------------|----------|-----------------------------------|
| Job openings CRUD | HR_ADMIN | Status: OPEN/CLOSED/ON_HOLD       |
| Candidates        | HR_ADMIN | Linked to opening; stage tracking |
| Public apply      | varies   | Tenant-scoped candidate create    |

### 7.17 Performance ( `/performance/*` )

**What it does:** Performance cycles, reviews, and goals per employee.

**Why it matters:** Closes the loop between attendance/timesheets and employee development.

**How it works:** Models `PerformanceCycle` , `PerformanceReview` , `PerformanceGoal` with tenant isolation.

| Surface | Description                       |
|---------|-----------------------------------|
| Cycles  | Annual/quarterly review periods   |
| Reviews | Manager ↔ employee review records |
| Goals   | OKR-style targets with status     |

### 7.18 Assets ( `/assets/*` )

**What it does:** Company asset inventory and employee asset requests.

**Why it matters:** Tracks laptops, badges, and equipment assigned to employees.

**How it works:** `Asset` master list + `AssetRequest` workflow (PENDING/APPROVED/DENIED).

### 7.19 Announcements ( `/announcements/*` )

**What it does:** Company-wide or channel-based announcements with read events.

**Why it matters:** Internal comms without email overload.

**How it works:** `AnnouncementChannel` → `Announcement` → `AnnouncementEvent` for delivery tracking.

### 7.20 Billing ( /billing/\* )

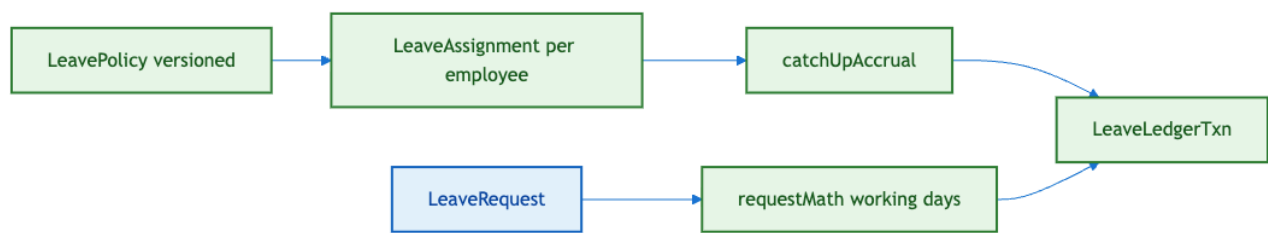
- What it does:** SaaS subscription billing surfaces for tenant admins.
- Why it matters:** SUPER\_ADMIN billing screens in settings UI.
- How it works:** `billing.service.js` returns plan/invoice shapes (integration-ready).

### 7.21 Logs ( /logs )

- What it does:** Query persisted application log entries ( `LogEntry` model).
- Why it matters:** Operational debugging beyond request-level Pino logs.
- How it works:** Filter by level (ERROR/WARN/INFO/DEBUG), paginated list.

### 7.22 Leave Engine ( /leave-engine/\* ) — Policy Engine Detail

| Component    | File                                     | Role                           |
|--------------|------------------------------------------|--------------------------------|
| Accrual      | <code>leave/engine/accrual.js</code>     | Monthly catch-up, tenure tiers |
| Proration    | <code>leave/engine/proration.js</code>   | Join/exit partial periods      |
| Ledger       | <code>leave/engine/ledger.js</code>      | Balance from txn sum           |
| Encashment   | <code>leave/engine/encashment.js</code>  | Year-end encash rules          |
| Year-end     | <code>leave/engine/yearEnd.js</code>     | Carry-forward / lapse          |
| Request math | <code>leave/engine/requestMath.js</code> | Working days in leave span     |
| Packs        | <code>leave/data/leavePacks.js</code>    | Country starter policies       |



### 7.23 Payout ( /payroll/me/payout-methods )

- What it does:** Employee bank account / payout method management with country-specific validation.
- Why it matters:** Payroll disbursement requires validated IBAN/account numbers per country.

**How it works:** payout/ submodule — bankSchemaCatalog.js , bankChecksums.js , bankFieldValidation.js , isoCountries.js .

## 7.24 Holidays Policy ( /holidays/policy/\* )

| Feature           | Description                                                    |
|-------------------|----------------------------------------------------------------|
| Resolved calendar | holidayResolver.service.js merges tenant + optional selections |
| Observed dates    | utils/observedDates.js — weekend shift rules                   |
| Applicability     | utils/applicability.js — location/department filters           |
| ICS import        | utils/icsParser.js for bulk holiday upload                     |

## 7.25 Timesheets Config ( /timesheets/config/\* )

| Feature        | Route                  | Roles                      |
|----------------|------------------------|----------------------------|
| Period locks   | /timesheets/locks      | HR create/delete, MGR read |
| Approval chain | settings blob          | HR configure               |
| Rate config    | budget/rate math utils | HR                         |
| Templates      | TimesheetTemplate      | HR                         |

## 7.26 Auth Submodule Map

| Submodule      | Files                                   | Concern                               |
|----------------|-----------------------------------------|---------------------------------------|
| Core auth      | auth.service.js ,<br>auth.repository.js | Login, refresh, sessions              |
| Password reset | passwordReset.service.js                | Token hash, TTL, rate limit           |
| OTP / MFA      | otp.service.js , otp.repository.js      | Challenge lifecycle                   |
| Invitations    | invitation.service.js                   | Employee onboarding emails            |
| Cookies        | auth.cookies.js                         | httpOnly accessToken +<br>ems_session |
| Policy         | auth.policy.js                          | MFA requirement rules                 |

## 7.27 Settings & Integrations ( /settings/\* )

| Area            | Key endpoints                                            |
|-----------------|----------------------------------------------------------|
| Tenant profile  | GET/PATCH /settings/tenant — logo, work week, timezone   |
| Email templates | GET/PATCH /settings/email-templates/:type                |
| Roles display   | GET/PATCH /settings/roles-permissions (SUPER_ADMIN)      |
| Integrations    | storage status, email provider — integrations.service.js |
| Security        | mfa_policy in tenant security settings                   |

## 7.28 Module → Prisma Model Quick Map

| Module        | Primary models                                                     |
|---------------|--------------------------------------------------------------------|
| auth          | User, Session, PasswordResetToken, OtpChallenge, UserInvitation    |
| employees     | Employee, EmployeeDocument                                         |
| departments   | Department                                                         |
| attendance    | AttendanceRecord, AttendanceRegularizationRequest                  |
| leave         | LeaveType, LeaveBalance, LeaveRequest, LeavePolicy, LeaveLedgerTxn |
| holidays      | Holiday, HolidayPolicy, HolidayOptionalSelection                   |
| timesheets    | Timesheet, TimeEntry, TimesheetProject, TimesheetTask              |
| payroll       | PayrollRun, Payslip, PayGroup, StatutoryPack, EmployeeSalary       |
| payout        | PayoutMethod, PayoutApproval, CountryBankSchema                    |
| notifications | Notification                                                       |
| auditLogs     | AuditLog                                                           |
| export        | ExportJob                                                          |
| reports       | ScheduledReport, ReportExport                                      |
| recruitment   | JobOpening, Candidate                                              |
| performance   | PerformanceCycle, PerformanceReview, PerformanceGoal               |
| assets        | Asset, AssetRequest                                                |
| announcements | Announcement, AnnouncementChannel                                  |



## 8. Database Schema Overview

**In simple terms:** Prisma models mirror PostgreSQL tables — 40+ entities scoped by `tenantId` .  
*Schema relationships drive how services join data; no raw SQL in this codebase.*

### 8.1 Model Inventory (78 Prisma Models)

| #  | Model              | Domain | Purpose                               |
|----|--------------------|--------|---------------------------------------|
| 1  | Tenant             | Core   | Multi-tenant root                     |
| 2  | User               | Core   | Login credentials + memberType        |
| 3  | Employee           | Core   | Employment profile, manager hierarchy |
| 4  | Department         | Core   | Org tree (parentId, depth)            |
| 5  | Role               | RBAC   | Named roles (future fine-grained)     |
| 6  | Permission         | RBAC   | Permission keys per module            |
| 7  | RolePermission     | RBAC   | Role ↔ Permission join                |
| 8  | UserRole           | RBAC   | User ↔ Role join                      |
| 9  | Session            | Auth   | Refresh token + familyId              |
| 10 | PasswordResetToken | Auth   | Hashed reset tokens                   |
| 11 | OtpChallenge       | Auth   | MFA OTP challenges                    |
| 12 | UserInvitation     | Auth   | Employee invite tokens                |

| #  | Model                           | Domain     | Purpose                           |
|----|---------------------------------|------------|-----------------------------------|
| 13 | EmployeeDocument                | Documents  | Cloudinary file metadata          |
| 14 | AttendanceRecord                | Attendance | Daily check-in/out                |
| 15 | AttendanceRegularizationRequest | Attendance | Correction workflow               |
| 16 | LeaveType                       | Leave      | Tenant leave categories           |
| 17 | LeaveBalance                    | Leave      | Cached balance per type           |
| 18 | LeaveRequest                    | Leave      | Approval workflow                 |
| 19 | LeavePolicy                     | Leave      | Versioned policy config           |
| 20 | LeaveAssignment                 | Leave      | Policy → employee mapping         |
| 21 | LeaveLedgerTxn                  | Leave      | Append-only balance ledger        |
| 22 | CompOffRequest                  | Leave      | Compensatory off                  |
| 23 | Holiday                         | Holidays   | Calendar dates                    |
| 24 | HolidayPolicy                   | Holidays   | Optional holiday rules            |
| 25 | HolidayOptionalSelection        | Holidays   | Employee selections               |
| 26 | Resignation                     | HR         | Resignation workflow (model only) |
| 27 | AuditLog                        | Audit      | Immutable mutation log            |
| 28 | LogEntry                        | Audit      | Application logs                  |
| 29 | Notification                    | Comms      | In-app notifications              |
| 30 | SavedView                       | UX         | Saved list filters                |
| 31 | Setting                         | Config     | Key-value tenant store            |
| 32 | ScheduledReport                 | Reports    | Cron schedule config              |
| 33 | ReportExport                    | Reports    | Export job tracking               |
| 34 | TenantConfig                    | Config     | Work week, fiscal year            |
| 35 | EmailTemplate                   | Config     | Custom email bodies               |
| 36 | ExportJob                       | Export     | Async export status               |
| 37 | SalaryComponent                 | Payroll    | Earning/deduction definitions     |

| #  | Model                 | Domain      | Purpose                       |
|----|-----------------------|-------------|-------------------------------|
| 38 | PayGroup              | Payroll     | Employee grouping             |
| 39 | PayGroupComponent     | Payroll     | Components in group           |
| 40 | EmployeeSalary        | Payroll     | Employee pay assignment       |
| 41 | PayrollRun            | Payroll     | Calculation batch             |
| 42 | Payslip               | Payroll     | Per-employee result           |
| 43 | LegalEntity           | Payroll     | Company registration unit     |
| 44 | StatutoryPack         | Payroll     | Country tax/contribution JSON |
| 45 | TaxDeclaration        | Payroll     | Employee tax regime choice    |
| 46 | EmployeeLoan          | Payroll     | Salary advance / loan         |
| 47 | PayrollInput          | Payroll     | Ad-hoc run inputs             |
| 48 | PayCalendar           | Payroll     | Pay period calendar           |
| 49 | OpeningBalance        | Payroll     | Migration opening balances    |
| 50 | HistoricalPayslip     | Payroll     | Imported legacy payslips      |
| 51 | MigrationStatus       | Payroll     | Data migration tracking       |
| 52 | ReimbursementCategory | Payroll     | Expense categories            |
| 53 | ReimbursementClaim    | Payroll     | Employee claims               |
| 54 | Garnishment           | Payroll     | Court-ordered deductions      |
| 55 | ContractorInvoice     | Payroll     | Contractor payments           |
| 56 | PaymentBatch          | Payroll     | Disbursement batch            |
| 57 | PayrollEvent          | Payroll     | Run audit events              |
| 58 | PayslipTemplate       | Payroll     | PDF template config           |
| 59 | PayoutMethod          | Payout      | Employee bank details         |
| 60 | PayoutApproval        | Payout      | Payout change approval        |
| 61 | CountryBankSchema     | Payout      | Per-country field rules       |
| 62 | JobOpening            | Recruitment | Open positions                |

| #  | Model               | Domain      | Purpose                 |
|----|---------------------|-------------|-------------------------|
| 63 | Candidate           | Recruitment | Applicants              |
| 64 | PerformanceCycle    | Performance | Review periods          |
| 65 | PerformanceReview   | Performance | Review records          |
| 66 | PerformanceGoal     | Performance | Employee goals          |
| 67 | Asset               | Assets      | Company assets          |
| 68 | AssetRequest        | Assets      | Employee requests       |
| 69 | AnnouncementChannel | Comms       | Announcement channels   |
| 70 | Announcement        | Comms       | Posts                   |
| 71 | AnnouncementEvent   | Comms       | Read/delivery events    |
| 72 | TimesheetProject    | Timesheets  | Projects                |
| 73 | TimesheetTask       | Timesheets  | Tasks under projects    |
| 74 | Timesheet           | Timesheets  | Weekly sheet header     |
| 75 | TimeEntry           | Timesheets  | Daily hour rows         |
| 76 | TimesheetSettings   | Timesheets  | Tenant timesheet config |
| 77 | TimesheetTemplate   | Timesheets  | Entry templates         |

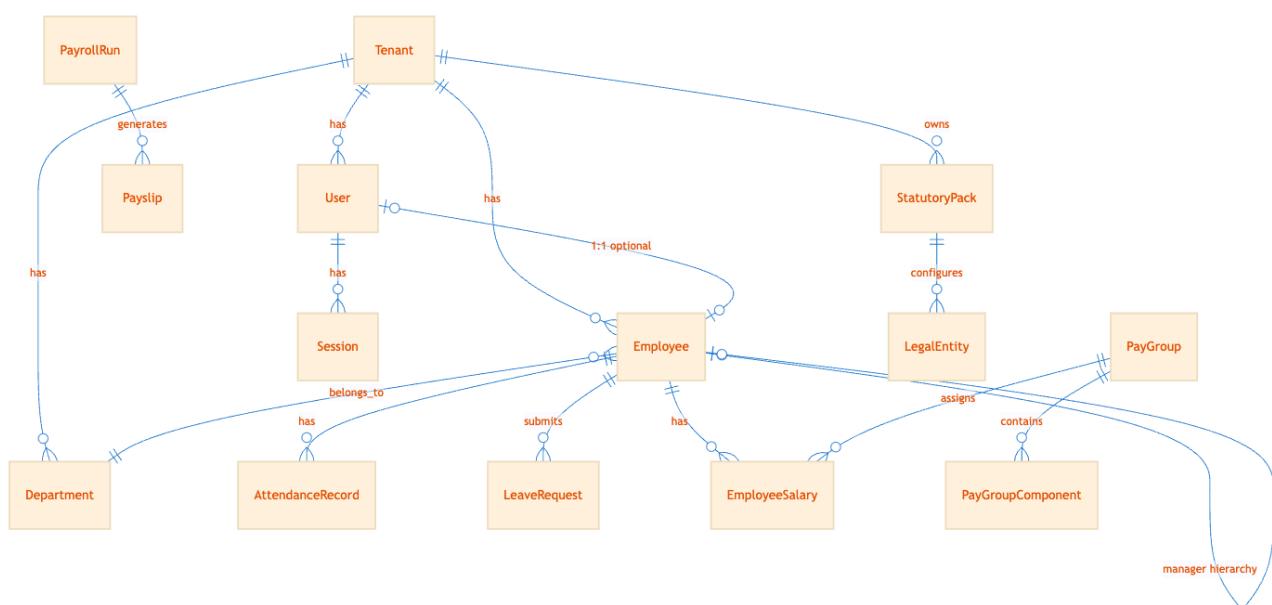
**Note:** `Resignation` model exists; dedicated API module not yet wired. RBAC models exist but `authorize()` uses `memberType` enum today.

### 8.1b Domain Grouping (Summary Table)

| Domain     | Model count | Key models                                                |
|------------|-------------|-----------------------------------------------------------|
| Core       | 4           | Tenant, User, Employee, Department                        |
| RBAC       | 4           | Role, Permission, RolePermission, UserRole                |
| Auth       | 5           | Session, PasswordResetToken, OtpChallenge, UserInvitation |
| Attendance | 2           | AttendanceRecord, AttendanceRegularizationRequest         |
| Leave      | 7           | LeaveType, LeaveRequest, LeaveLedgerTxn, ...              |

| Domain        | Model count | Key models                                           |
|---------------|-------------|------------------------------------------------------|
| Holidays      | 3           | Holiday, HolidayPolicy, HolidayOptionalSelection     |
| Payroll       | 22          | PayrollRun, Payslip, StatutoryPack, PayGroup, ...    |
| Payout        | 3           | PayoutMethod, PayoutApproval, CountryBankSchema      |
| Timesheets    | 6           | Timesheet, TimeEntry, TimesheetProject, ...          |
| Recruitment   | 2           | JobOpening, Candidate                                |
| Performance   | 3           | PerformanceCycle, PerformanceReview, PerformanceGoal |
| Assets        | 2           | Asset, AssetRequest                                  |
| Announcements | 3           | Announcement, AnnouncementChannel, AnnouncementEvent |
| Platform      | 6           | AuditLog, Notification, ExportJob, Setting, ...      |

## 8.2 Core Entity Relationships



## 8.3 Multi-Tenant Isolation

Every model with business data includes:

```
tenantId String
tenant    Tenant @relation(fields: [tenantId], references: [id])
```

All repository queries MUST include `where: { tenantId }` — enforced by convention, not database RLS.

## 9. Payroll Engine

**In simple terms:** Payroll computes gross → statutory deductions → tax → net per employee per run.  
*Highest-risk domain — calculations are data-driven via StatutoryPack JSON, not hardcoded country logic.*

### 9.1 Overview

The payroll engine supports **multi-country statutory compliance** through configurable statutory packs. Calculation is config-driven — no hardcoded country logic in the core loop.

### 9.2 Supported Countries (Built-in Metadata)

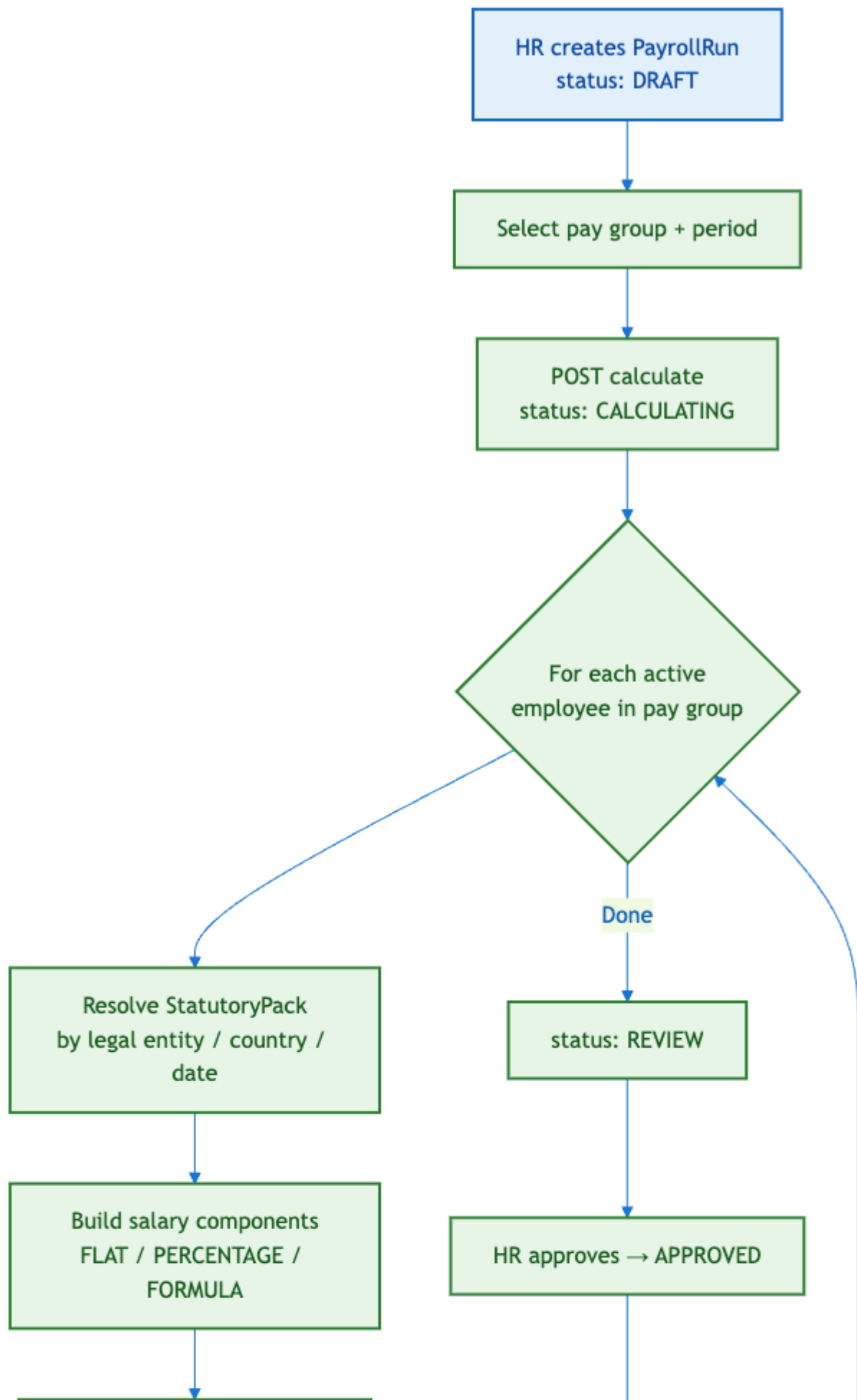
| Code | Country        | Currency | Fiscal Year Start |
|------|----------------|----------|-------------------|
| IN   | India          | INR      | April (month 4)   |
| US   | United States  | USD      | January           |
| GB   | United Kingdom | GBP      | April             |
| SA   | Saudi Arabia   | SAR      | January           |
| AE   | UAE            | AED      | January           |
| VN   | Vietnam        | VND      | January           |
| SG   | Singapore      | SGD      | January           |
| CA   | Canada         | CAD      | January           |

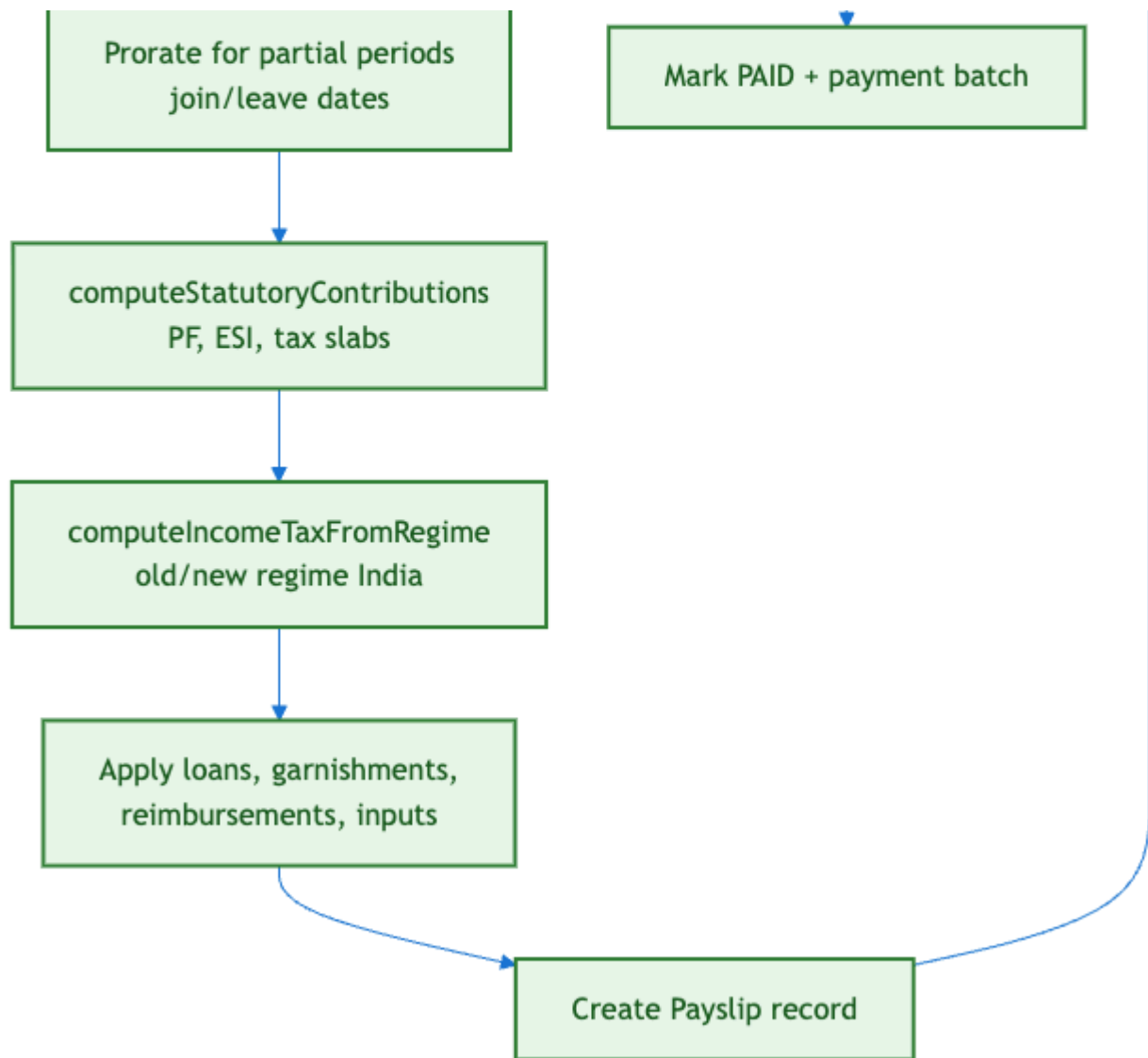
Additional countries (e.g., **KW/KWD** for `kwd-litmus-001` tenant) are supported via tenant-specific

StatutoryPack and LegalEntity configuration without requiring code changes.

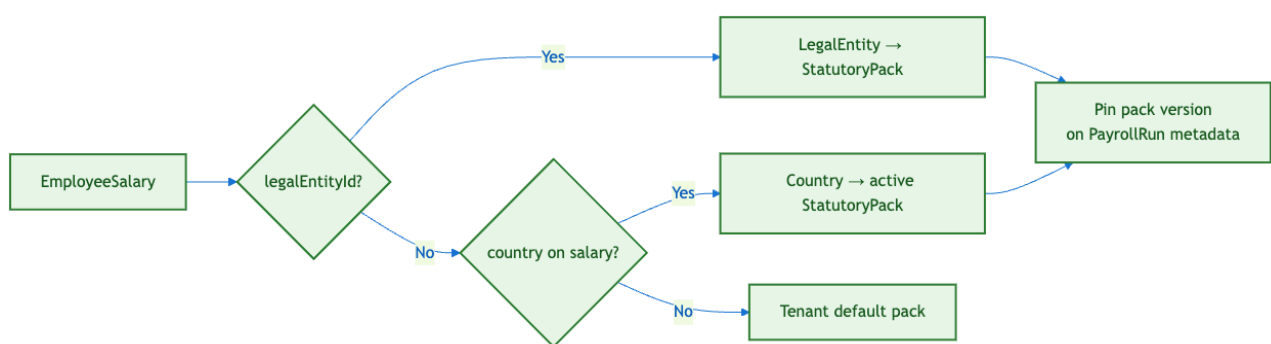
**9.3 Payroll Run Flow**







## 9.4 Statutory Pack Resolution



**India-specific:** PF, ESI, professional tax, income tax slabs (old/new regime), HRA exemptions.

**KWD / Gulf:** No built-in `SUPPORTED_COUNTRIES` entry — requires tenant-seeded `StatutoryPack` with country `KW` and appropriate contribution schemes.

## 9.5 Component Calculation Types

| Type       | Behavior                                                   |
|------------|------------------------------------------------------------|
| FLAT       | Fixed monthly amount                                       |
| PERCENTAGE | % of <code>basisCode</code> component (e.g., 40% of BASIC) |
| FORMULA    | Evaluated expression referencing computed component codes  |

## 9.6 Payslip Generation

After run approval, payslips are generated per employee with: - Earnings, deductions, employer contributions breakdown - Statutory line items from pinned pack version - YTD accumulators for tax and statutory caps - Export via `GET /payroll/runs/:id/payslips/export`

## 10. External Integrations

**In simple terms:** Email (Resend), file storage (Cloudinary), and optional Redis (BullMQ) live outside the monolith.

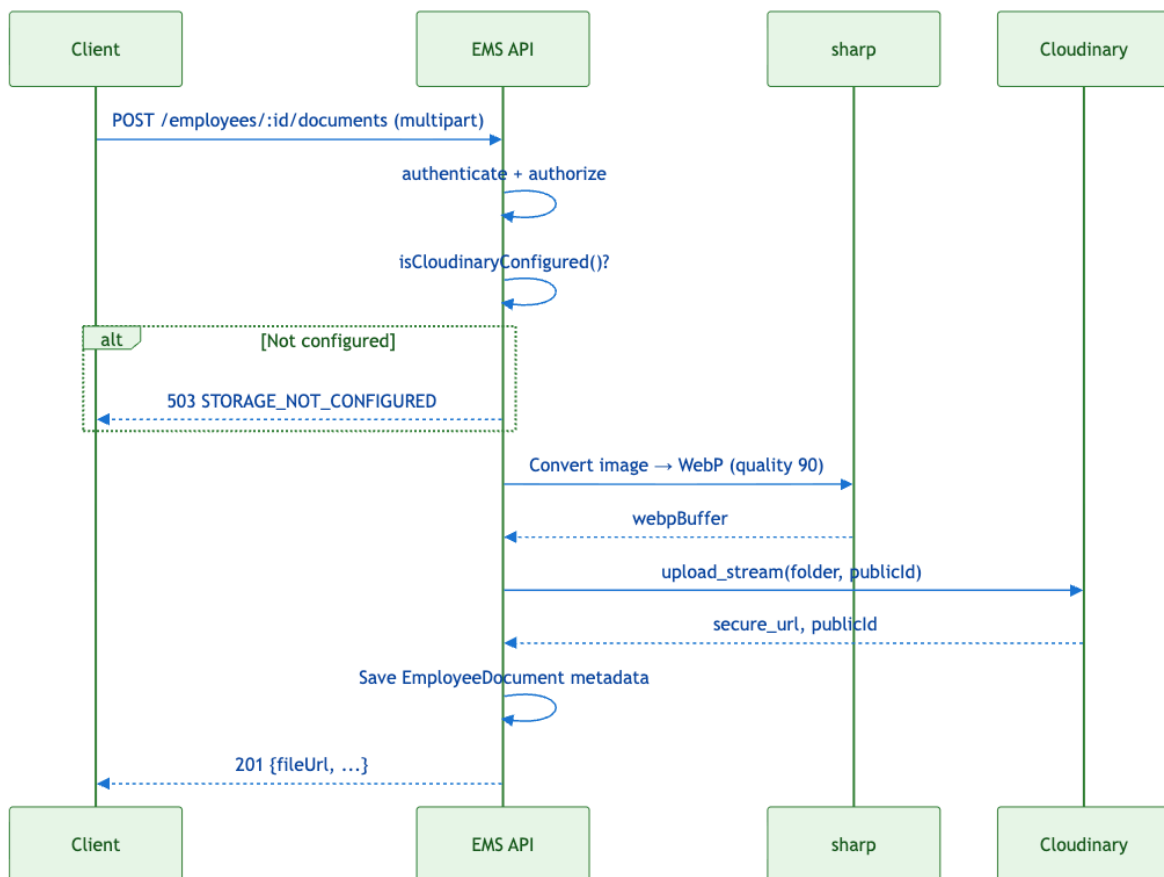
*Each integration degrades gracefully when env vars are missing.*

## 10.1 Resend (Email)



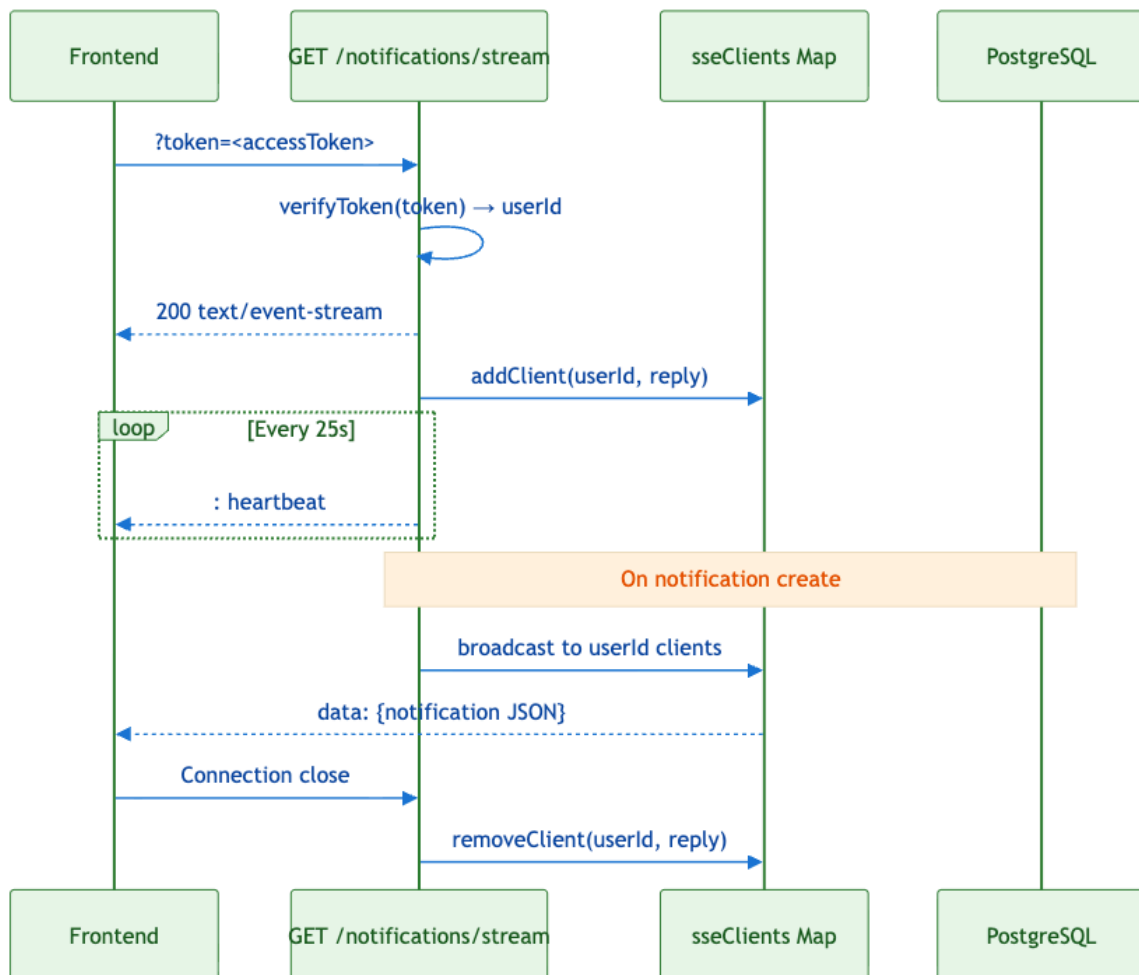
- Primary provider for OTP, password reset, employee invitations
- Fallback: SMTP (Brevo) configurable via env vars
- Rate limits on forgot-password and OTP endpoints

## 10.2 Cloudinary (File Upload)



**Upload targets:** - Employee documents ( `POST /employees/:id/documents` ) - Employee photos ( `POST /employees/:id/photo` ) — always WebP - Tenant logo ( `PATCH /settings/tenant` with logo file)

### 10.3 Notifications SSE Stream



- Polling fallback: `GET /notifications?since=ISO`
- Unread badge: `GET /notifications/unread-count`
- Mark read: `PATCH /notifications/:id/read` (POST alias supported)

### 10.4 Render (Legacy Deployment)

| Field    | Value                                                             |
|----------|-------------------------------------------------------------------|
| Service  | Employee-management-system                                        |
| URL      | <code>https://employee-management-system-2b9q.onrender.com</code> |
| Database | Cross-account Render PostgreSQL (external URL)                    |
| Status   | Legacy — superseded by Hostinger for production traffic           |

### 10.5 Hostinger (Primary Production)

| Field          | Value                            |
|----------------|----------------------------------|
| API URL        | https://ems-api.saqibsaeed.cloud |
| VPS Path       | /opt/ems/app                     |
| Containers     | ems-backend , ems-postgres       |
| Deploy trigger | Push to main → GitHub Actions    |

## 11. Error Handling & Logging

**In simple terms:** All errors return a consistent JSON envelope via `errorResponse()` .  
*Pino logs every request; audit logs capture business mutations immutably.*

### 11.1 Response Envelope

Success:

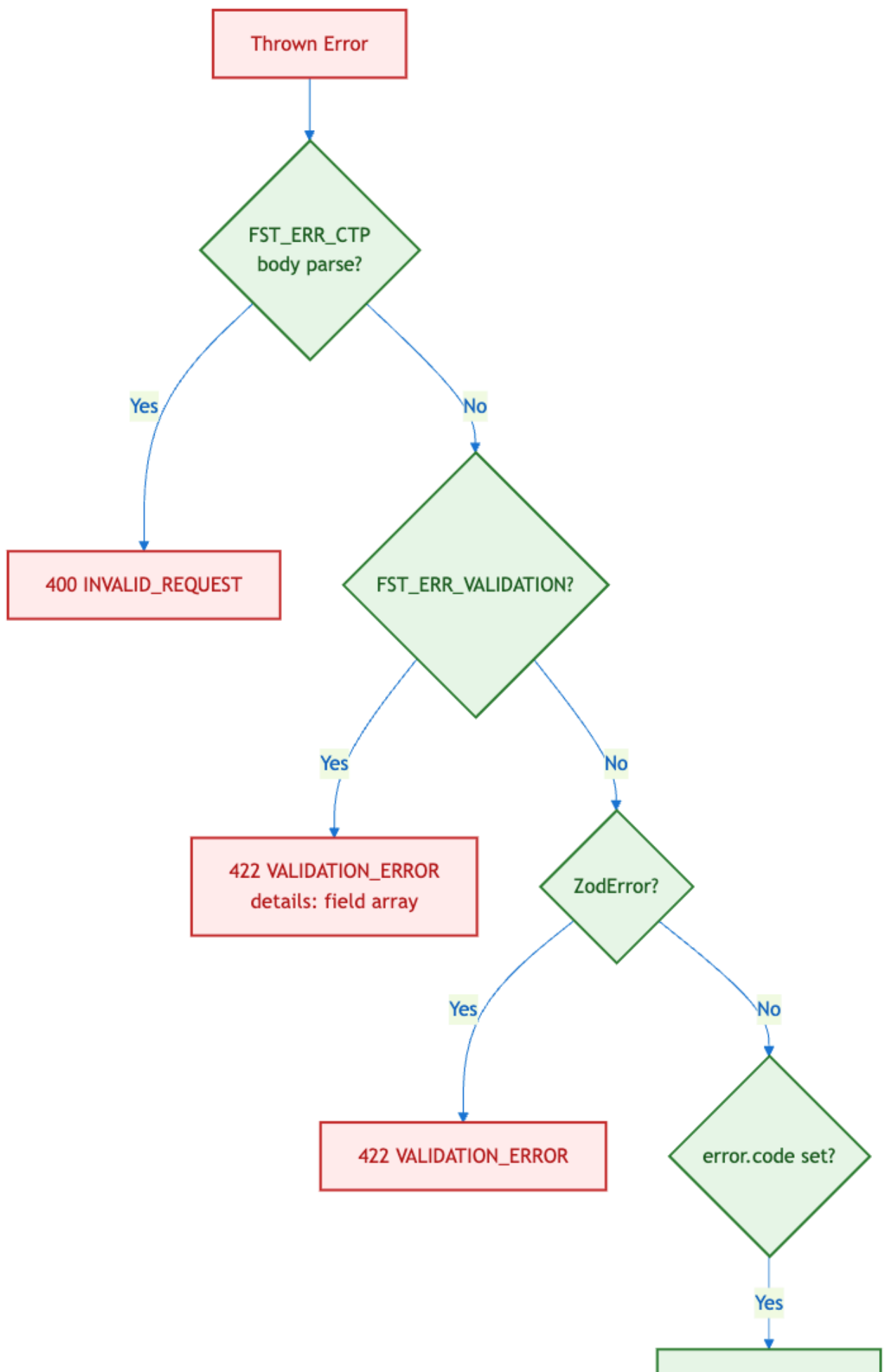
```
{ "success": true, "data": {}, "meta": {} }
```

Error:

```
{
  "success": false,
  "error": {
    "code": "MACHINE_READABLE_CODE",
    "message": "Human-readable message",
    "details": {},
    "requestId": "req-uuid"
  }
}
```

**11.2 Error Handler Pipeline**





error.statusCode  
default 500

### 11.3 HTTP Status Contract

| Situation                    | Status |
|------------------------------|--------|
| Success GET/PATCH/DELETE     | 200    |
| Success POST create          | 201    |
| MFA challenge / async        | 202    |
| Field validation             | 422    |
| Missing/invalid token        | 401    |
| Insufficient role            | 403    |
| Not found                    | 404    |
| Conflict (duplicate, cycle)  | 409    |
| Bad request / missing tenant | 400    |
| Server error                 | 500    |

### 11.4 Structured Logging

- **Pino** logger with request ID correlation
- `requestLogging.js` hook logs method, URL, status, duration
- `LogEntry` model for persisted application logs (ERROR/WARN/INFO/DEBUG)
- `AuditLog` for immutable entity mutation tracking

---

---

---

---

---

---

---

---

---

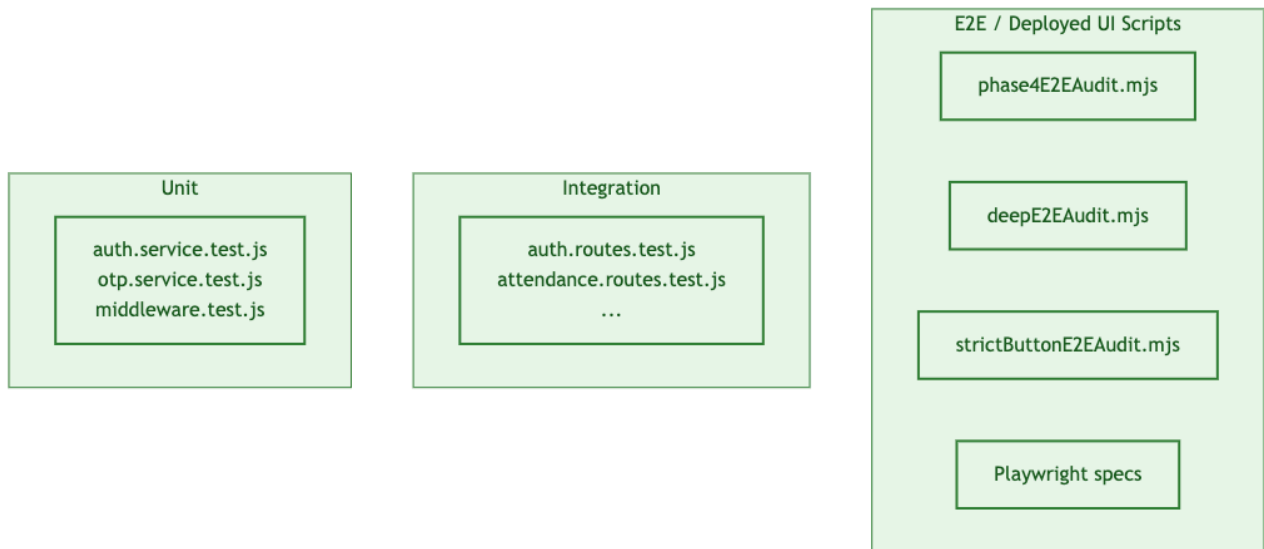
---

## 12. Testing Strategy

**In simple terms:** Unit, integration, and E2E tests guard regressions — integration tests use a local test DB only.

Never run `npm test` against production `DATABASE_URL`.

### 12.1 Test Pyramid



### 12.2 Test Categories

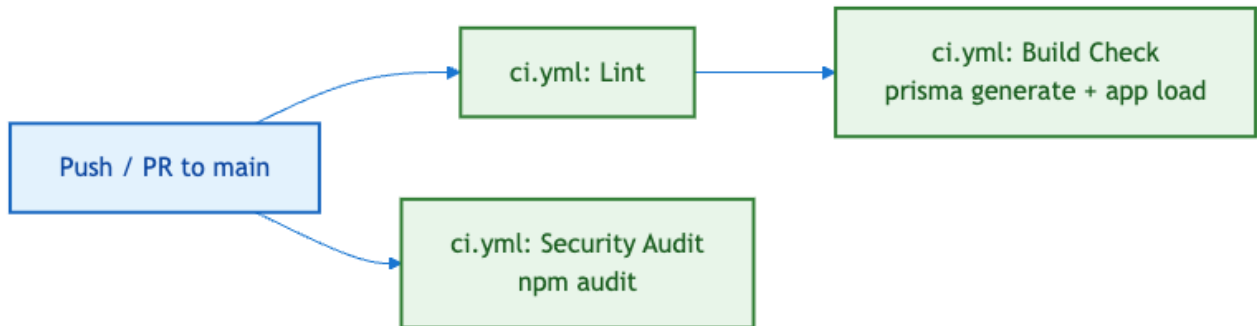
| Category      | Location / Command                                                | Scope                            |
|---------------|-------------------------------------------------------------------|----------------------------------|
| Smoke         | <code>npm run test:smoke</code>                                   | App bootstrap                    |
| Unit          | <code>tests/unit/</code>                                          | Service logic, utils, middleware |
| Integration   | <code>tests/integration/</code>                                   | Route-level with test DB         |
| Auth-specific | <code>npm run test:auth-me</code> , <code>test:auth-logout</code> | Session edge cases               |
| Deployed UI   | <code>npm run test:deployed-ui:*</code>                           | Live Hostinger UI clickthrough   |
| E2E Deep      | <code>npm run test:e2e:deep:*</code>                              | API matrix + CRUD probes         |
| Phase 4       | <code>npm run test:e2e:phase4:*</code>                            | Edge cases + exhaustive UI       |
| Playwright    | <code>npm run test:playwright:*</code>                            | Browser automation               |

### 12.3 Test Database Safety

`cleanDatabase()` in `tests/helpers.js` is guarded: - Requires `NODE_ENV=test` - `DATABASE_URL` must target `localhost`, `127.0.0.1`, or DB name containing `ems_test` - Prevents accidental

production data wipe

## 12.4 CI Pipeline

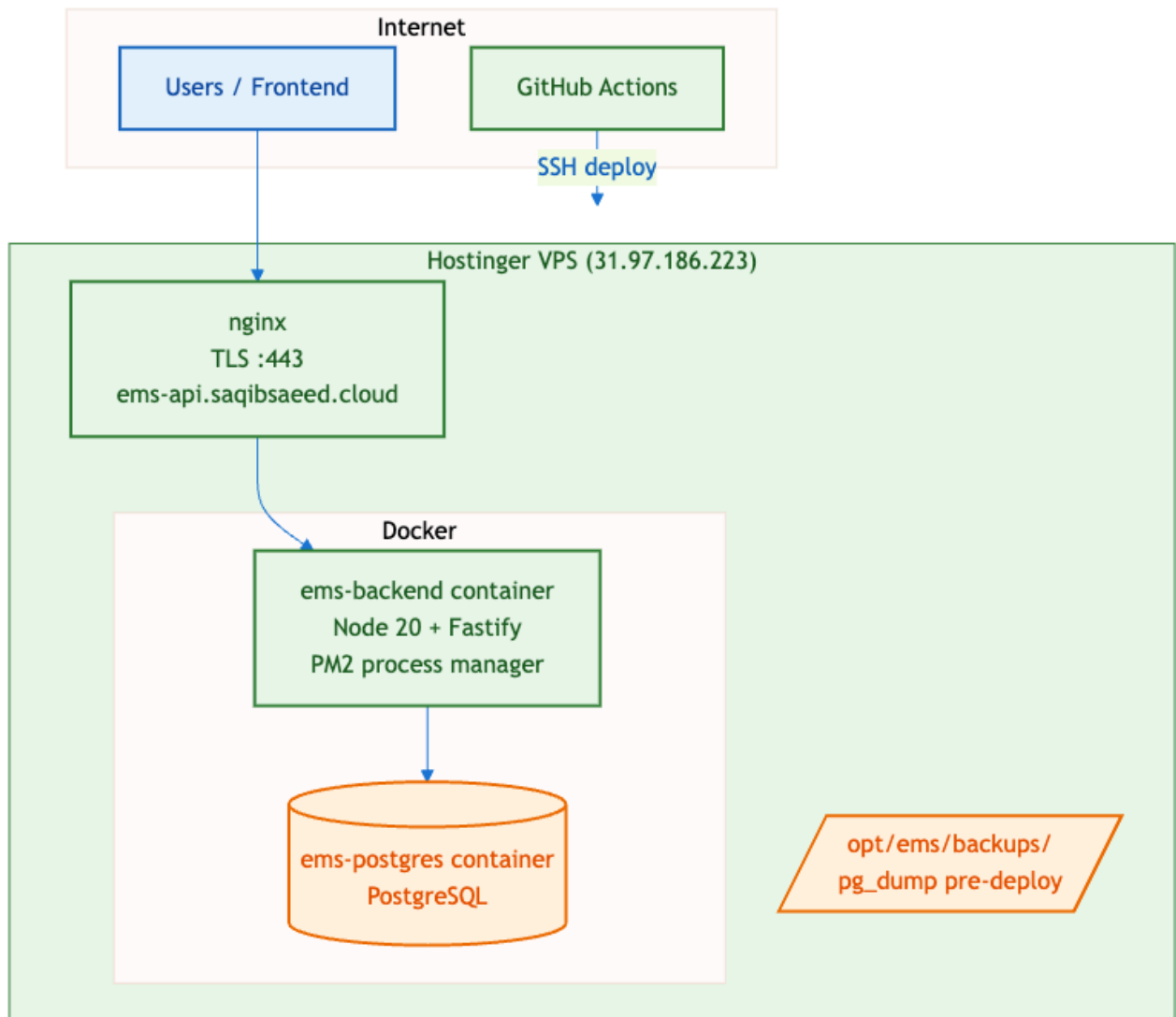


## 13. Deployment

**In simple terms:** Primary production runs on Hostinger VPS (Docker + nginx); Render is a legacy target.

*Deploys are automated via GitHub Actions with pre-deploy database backups.*

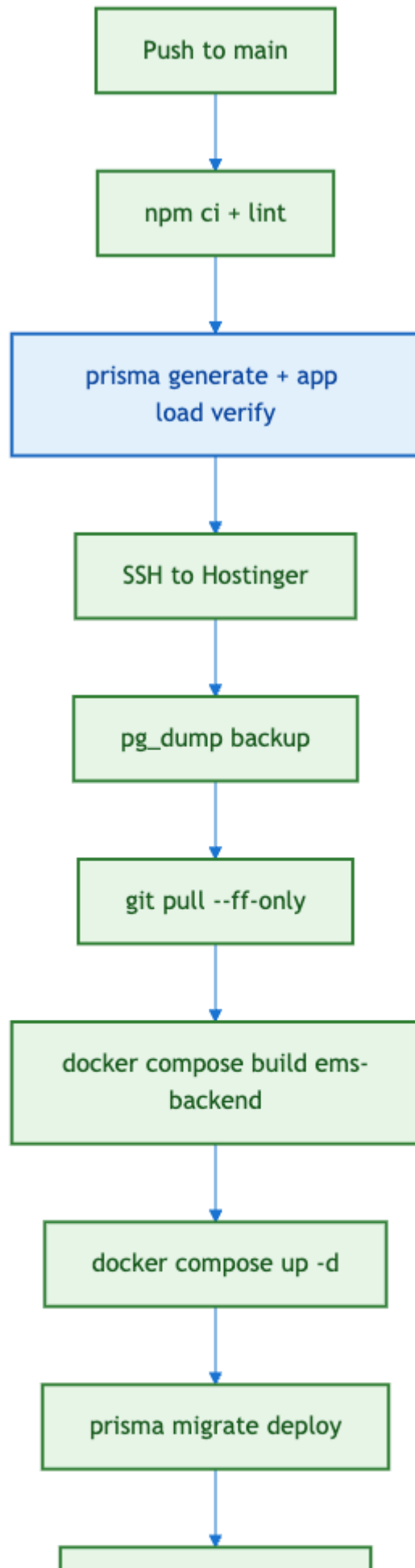
## 13.1 Hostinger VPS Architecture

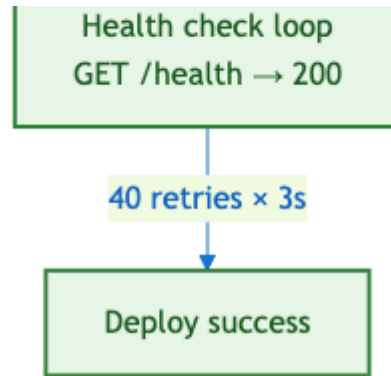


## 13.2 ASCII — Hostinger Directory Layout

```
/opt/ems/
├─ app/                # Git clone (origin/main)
│  └─ Dockerfile       # Box-only (not in upstream)
│     └─ .env           # Production secrets (box-only)
│        └─ docker-compose.yml
├─ backups/            # Pre-deploy pg_dump archives (keep 14)
└─ docker-compose.yml  # Orchestrates ems-backend + ems-postgres
```

### 13.3 CI/CD Pipeline (GitHub Actions → Hostinger)





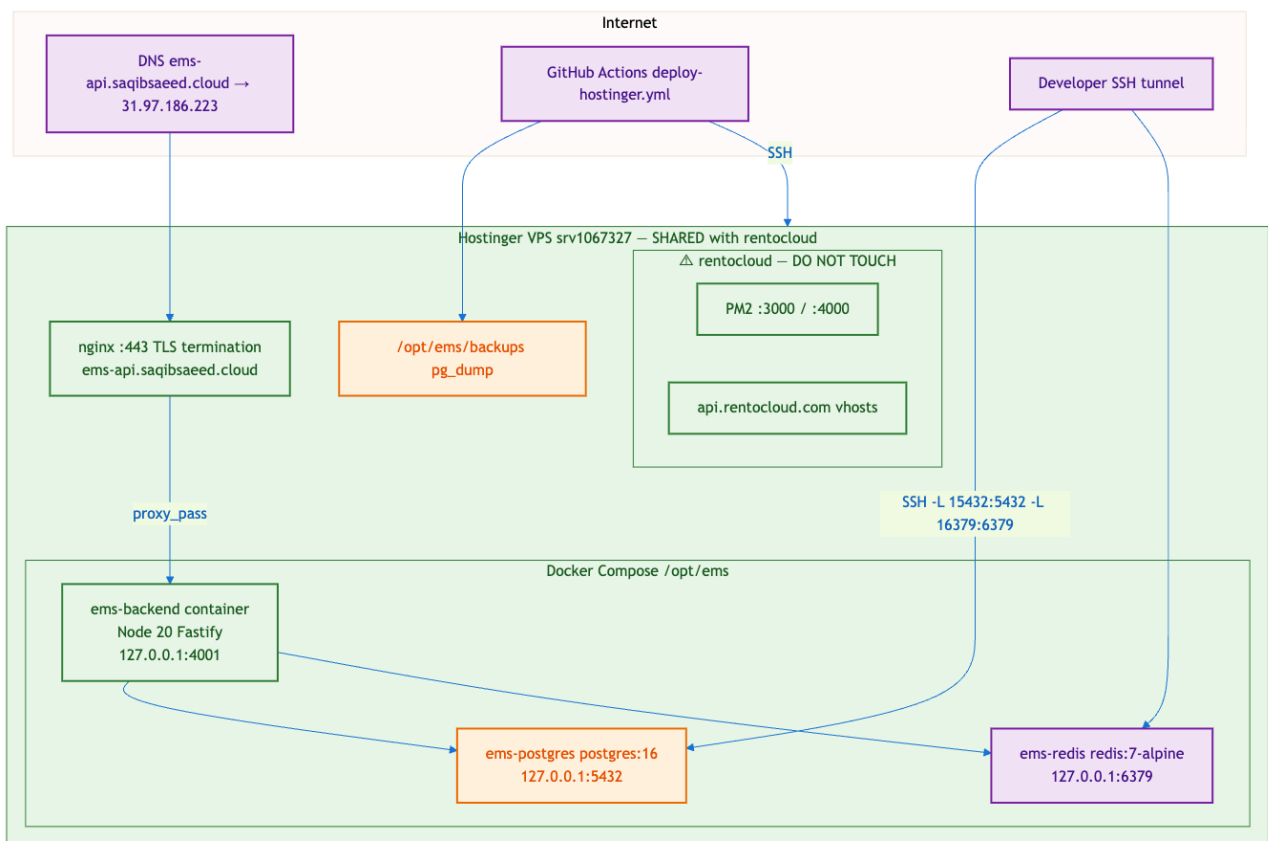
## 13.6 Hostinger Deep Dive (Production)

**What it does:** Primary production deployment on a shared Hostinger VPS running Docker + nginx.

**Why it matters:** Most live traffic and the UI team's integration target.

**How it works:** DNS → nginx TLS :443 → Docker ems-backend :4001 → Postgres + Redis on ems-net.

### 13.6.1 Full VPS Architecture





### 13.6.2 Port Mapping

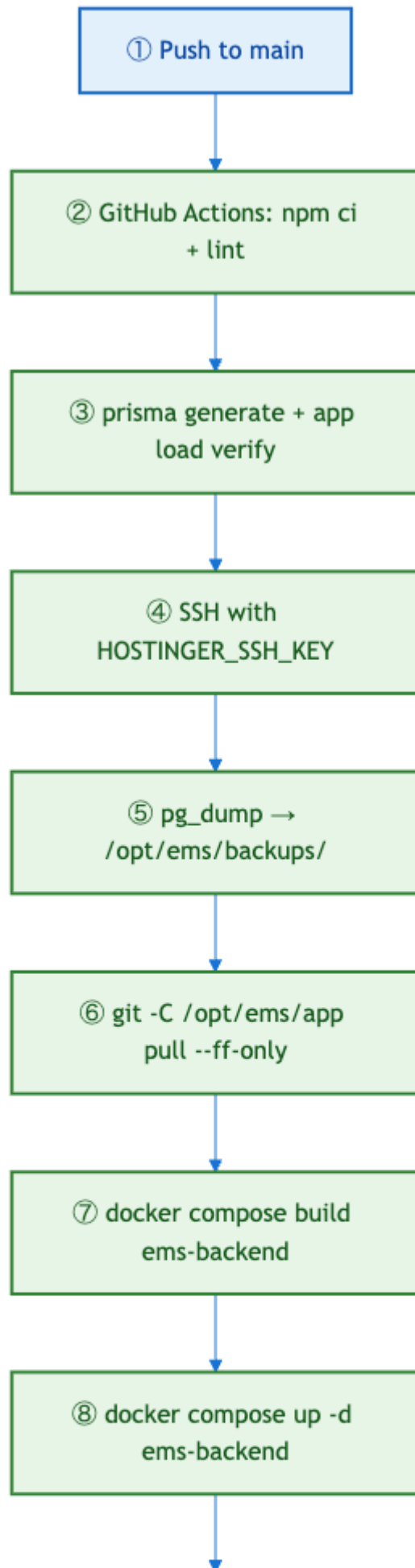
| Exposure      | Host      | Container         | Service                |
|---------------|-----------|-------------------|------------------------|
| Public 443    | nginx     | —                 | TLS + reverse proxy    |
| Loopback 4001 | 127.0.0.1 | ems-backend:3000  | Fastify API            |
| Loopback 5432 | 127.0.0.1 | ems-postgres:5432 | PostgreSQL             |
| Loopback 6379 | 127.0.0.1 | ems-redis:6379    | Redis (cache + BullMQ) |

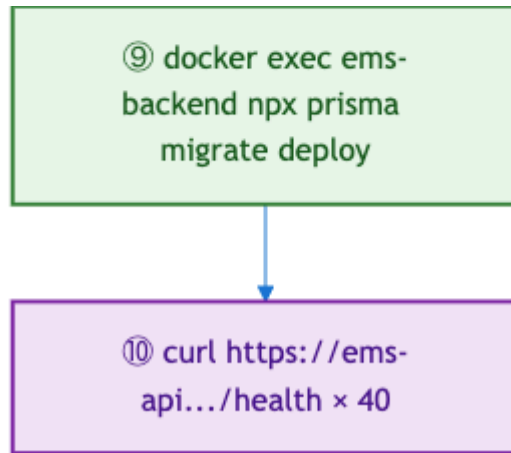
External clients **never** hit :4001 directly — only nginx on :443.

### 13.6.3 Directory Layout `/opt/ems/`

```
/opt/ems/
├─ docker-compose.yml          # Orchestrates backend + postgres + redis
├─ docker-compose.override.yml # Redis service (box-only)
├─ backups/
├─   └─ ems_predeploy_YYYYMMDD.sql.gz # Last 14 kept
├─ app/                        # Git clone of origin/main
│   ├── Dockerfile             # Box-only (untracked upstream)
│   ├── .dockerignore          # Box-only
│   ├── .env                   # Production secrets – NEVER commit
│   ├── src/
│   └─ prisma/
```

**13.6.4 SSH Deploy Flow (Step by Step)**





### 13.6.5 Environment Variables on VPS (categories)

| Category | Examples                                                         | Notes                    |
|----------|------------------------------------------------------------------|--------------------------|
| Core     | <code>NODE_ENV=production</code> , <code>PORT=3000</code>        | Inside container         |
| Database | <code>DATABASE_URL=postgresql://...@ems-postgres:5432/...</code> | Docker network hostname  |
| Redis    | <code>REDIS_URL=redis://redis:6379</code>                        | Enables BullMQ + cache   |
| JWT      | <code>JWT_SECRET</code>                                          | Non-default required     |
| Email    | <code>RESEND_API_KEY</code> , <code>RESEND_FROM</code>           | Live transactional email |
| Storage  | <code>CLOUDINARY_*</code>                                        | Document uploads         |
| CORS     | <code>CORS_ORIGIN</code>                                         | Frontend origins         |

### 13.6.6 Local Dev via SSH Tunnel

`scripts/startLocalHostingerStack.sh` :

1. Opens SSH tunnels `localhost:15432 → VPS:5432` and `localhost:16379 → VPS:6379`
2. Rewrites `DATABASE_URL` host to `127.0.0.1:15432`
3. Writes `/tmp/ems-tunnel.override.env`
4. Start: `node --env-file=.env --env-file=/tmp/ems-tunnel.override.env --watch src/server.js`

**Never** run migrations, seeds, or tests against tunneled production DB.

### 13.6.7 PM2 vs Docker — What Runs What

| Workload            | Runtime                          | Notes                                           |
|---------------------|----------------------------------|-------------------------------------------------|
| EMS API             | Docker <code>ems-backend</code>  | Built from <code>/opt/ems/app/Dockerfile</code> |
| EMS Postgres        | Docker <code>ems-postgres</code> | Isolated volume                                 |
| EMS Redis           | Docker <code>ems-redis</code>    | Payroll queue + hot config cache                |
| rentocloud API      | PM2 on host                      | <b>Do not modify</b>                            |
| rentocloud frontend | PM2 :3000                        | Separate nginx vhosts                           |

EMS and rentocloud share the **same VPS** but use **separate** process managers and nginx server blocks.

### 13.6.8 SSL/TLS at nginx

- Certificate: Let's Encrypt (or Hostinger-managed) for `ems-api.saqibsaeed.cloud`
- Termination at nginx — backend sees HTTP on loopback
- `X-Forwarded-Proto: https` passed to Fastify

### 13.6.9 Backup and Migration Strategy

| Step              | When                | Command                                                                                                                                                               |
|-------------------|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pre-deploy backup | Every GitHub deploy | <code>docker exec ems-postgres pg_dump ... \&lt;/td&gt;&lt;td style="background:#f5f9ff;padding:8px 10px;border:1px solid #e0e0e0;vertical-align:top;"&gt;gzip</code> |
| Retention         | Automatic           | Keep newest 14 backups                                                                                                                                                |
| Migrate           | After container up  | <code>docker exec ems-backend npx prisma migrate deploy</code>                                                                                                        |
| Rollback          | Manual              | Restore <code>.sql.gz</code> + pin previous image                                                                                                                     |

**Rule:** additive migrations only on production — never `migrate reset` or `db push --force-reset`.

### 13.6.10 Shared VPS Warnings (rentocloud)

| Do                                                       | Don't                                      |
|----------------------------------------------------------|--------------------------------------------|
| Edit EMS nginx vhost only                                | Restart rentocloud PM2 processes           |
| Use <code>docker compose</code> in <code>/opt/ems</code> | Change rentocloud nginx server blocks      |
| Deploy via GitHub Actions                                | Run <code>pm2 delete all</code> on the box |

| Do                                         | Don't                                     |
|--------------------------------------------|-------------------------------------------|
| SSH key <code>hostinger_ems_ed25519</code> | Share production <code>.env</code> in git |

### 13.7 Render vs Hostinger Comparison

| Dimension              | Render                                                    | Hostinger                                      |
|------------------------|-----------------------------------------------------------|------------------------------------------------|
| <b>Status</b>          | Legacy / test target                                      | <b>Primary production</b>                      |
| <b>URL</b>             | <code>employee-management-system-2b9q.onrender.com</code> | <code>ems-api.saqibsaeed.cloud</code>          |
| <b>Compute</b>         | Render Web Service (managed)                              | Docker on VPS (self-managed)                   |
| <b>Database</b>        | Render PostgreSQL (cross-account external URL)            | <code>ems-postgres</code> container            |
| <b>Redis</b>           | Not on Render deploy                                      | <code>ems-redis</code> live — BullMQ + cache   |
| <b>Deploy</b>          | Render auto-deploy on push                                | GitHub Actions → SSH                           |
| <b>TLS</b>             | Render-managed                                            | nginx on VPS                                   |
| <b>Cost model</b>      | PaaS per-service                                          | VPS flat (shared with rentocloud)              |
| <b>Cold start</b>      | Yes (free tier spin-down)                                 | No — always on                                 |
| <b>Connection pool</b> | <code>connection_limit=5</code> caused bell 500s          | Local pool in container                        |
| <b>Backups</b>         | Render DB backups                                         | <code>pg_dump</code> pre-deploy + retention 14 |
| <b>Local parity</b>    | External DATABASE_URL only                                | SSH tunnel script available                    |

### 13.4 Render (Legacy)

- Auto-deploy from GitHub on push (Render Web Service)
- Uses external PostgreSQL URL (cross-account)
- Still available as fallback; production frontend points to Hostinger

### 13.5 Local Development

```
docker compose up -d      # Local Postgres on :5432
npm run dev               # Fastify with --watch
npx prisma migrate deploy # Apply migrations
npm run db:seed           # Idempotent seed
```

---

---

---

---

---

---

---

---

---

---

## 14. Security Considerations

**In simple terms:** Defense in depth: Argon2id passwords, JWT rotation, rate limits, helmet headers, tenant isolation.  
*Review this before exposing new endpoints or changing auth flows.*

### 14.1 Authentication Security

| Control            | Implementation                                              |
|--------------------|-------------------------------------------------------------|
| Password hashing   | Argon2id (memory-hard)                                      |
| JWT secret         | Fail-closed in production if default                        |
| Refresh rotation   | Hash stored; family-ID reuse detection revokes all sessions |
| Session validation | Every request checks <code>Session.revokedAt</code>         |
| Cookie flags       | httpOnly, secure (production), SameSite                     |

### 14.2 Multi-Tenant Security

- All queries scoped by `tenantId` from resolved tenant (never from client body alone)
- JWT `tenantId` cross-checked against `request.tenant.id` in `authenticate()`
- Subdomain slug validation rejects `www` , `api` , `app`

### 14.3 Input Validation

- Zod schemas in services for business validation
- Fastify JSON Schema for route-level validation (422 responses)
- Tolerant JSON parser for empty-body POSTs (frontend axios compatibility)
- 10 MB multipart file size limit

### 14.4 HTTP Security Headers

- `@fastify/helmet` — CSP, X-Frame-Options, etc.
- `@fastify/rate-limit` — global rate limiting
- CORS restricted to configured origins

### 14.5 Data Protection

- Soft delete on Employee ( `deletedAt` ) — no hard delete by default
- Audit logs immutable (append-only)
- Password reset tokens hashed (SHA-256) with TTL
- OTP challenges rate-limited with attempt caps

---

---

---

---

---

---

---

---

---

---

## 15. Known Issues (E2E Audits)

**In simple terms:** Documented gaps found by automated UI/API audits — not blockers but tracked honestly.

*Check here before assuming a wireframe feature is fully implemented.*

A comprehensive E2E audit was conducted in July 2026. Full details: `docs/E2E_BACKEND_ISSUES.md` .



### Summary (28 documented issues)

| ID | Severity | Issue                                             | Root Cause                                       |
|----|----------|---------------------------------------------------|--------------------------------------------------|
| 1  | P1       | SUPER_ADMIN attendance returns NO_EMPLOYEE_RECORD | Admin user has no linked Employee record         |
| 2  | P2       | SUPER_ADMIN payout-methods 400                    | Same — /me/* requires employeeId                 |
| 3  | P1       | KWD tenant timesheets — salary 404                | Missing salary config for admin employee         |
| 4  | P2       | Storage integration provider: undefined           | Response shape mismatch with frontend            |
| —  | —        | KWD tenant no StatutoryPack                       | Data gap — blocks payroll for KW country         |
| —  | —        | MFA blocker (resolved)                            | Test users had mfaEnabled; disabled on Hostinger |

## Recommendations

1. Add employee record for SUPER\_ADMIN or return admin-scoped fallback on attendance endpoints
2. Seed salary + statutory pack for `kwd-litmus-001` tenant
3. Align `GET /settings/integrations/storage` response with frontend contract
4. Continue phase4 E2E scripts on every release ( `npm run test:e2e:phase4` )

[illegible]

## 16. API Reference Overview

**In simple terms:** Canonical contract lives in Swagger ( /docs ) and docs/API\_MAPPING.md .  
*This section summarizes route groups; use API\_MAPPING for field-level shapes.*

### 16.1 Interactive Documentation

| Resource                | URL                                                     |
|-------------------------|---------------------------------------------------------|
| Swagger UI              | <code>https://ems-api.saqibsaeed.cloud/docs</code>      |
| OpenAPI JSON            | <code>https://ems-api.saqibsaeed.cloud/docs/json</code> |
| API Mapping (canonical) | <code>docs/API_MAPPING.md</code> in repository          |

### 16.2 Response Envelope (All Endpoints)

All endpoints return the standard envelope documented in `docs/API_MAPPING.md` . Date format: `YYYY-MM-DD` for all input fields (except `holidayDate` which rejects full ISO).

### 16.3 Quick Test

```
curl -X POST https://ems-api.saqibsaeed.cloud/api/v1/auth/login \  
-H "Content-Type: application/json" \  
-H "X-Tenant-Key: acme-corp-001" \  
-d '{"email":"hr@acme.test","password":"Password123!"}'
```

### 16.4 Auth Headers After Login

| Context              | Headers                                                                   |
|----------------------|---------------------------------------------------------------------------|
| Browser              | Cookies auto-send ( <code>accessToken</code> , <code>ems_session</code> ) |
| Swagger/Postman      | <code>Authorization: Bearer &lt;accessToken&gt;</code>                    |
| Tenant (first login) | <code>X-Tenant-Key: acme-corp-001</code> (optional after JWT issued)      |

---

---

---

## 17. End-to-End Workflow Diagrams

**In simple terms:** Visual walkthroughs of major business flows with numbered steps.

*Blue = client, green = API, orange = database, purple = external services.*

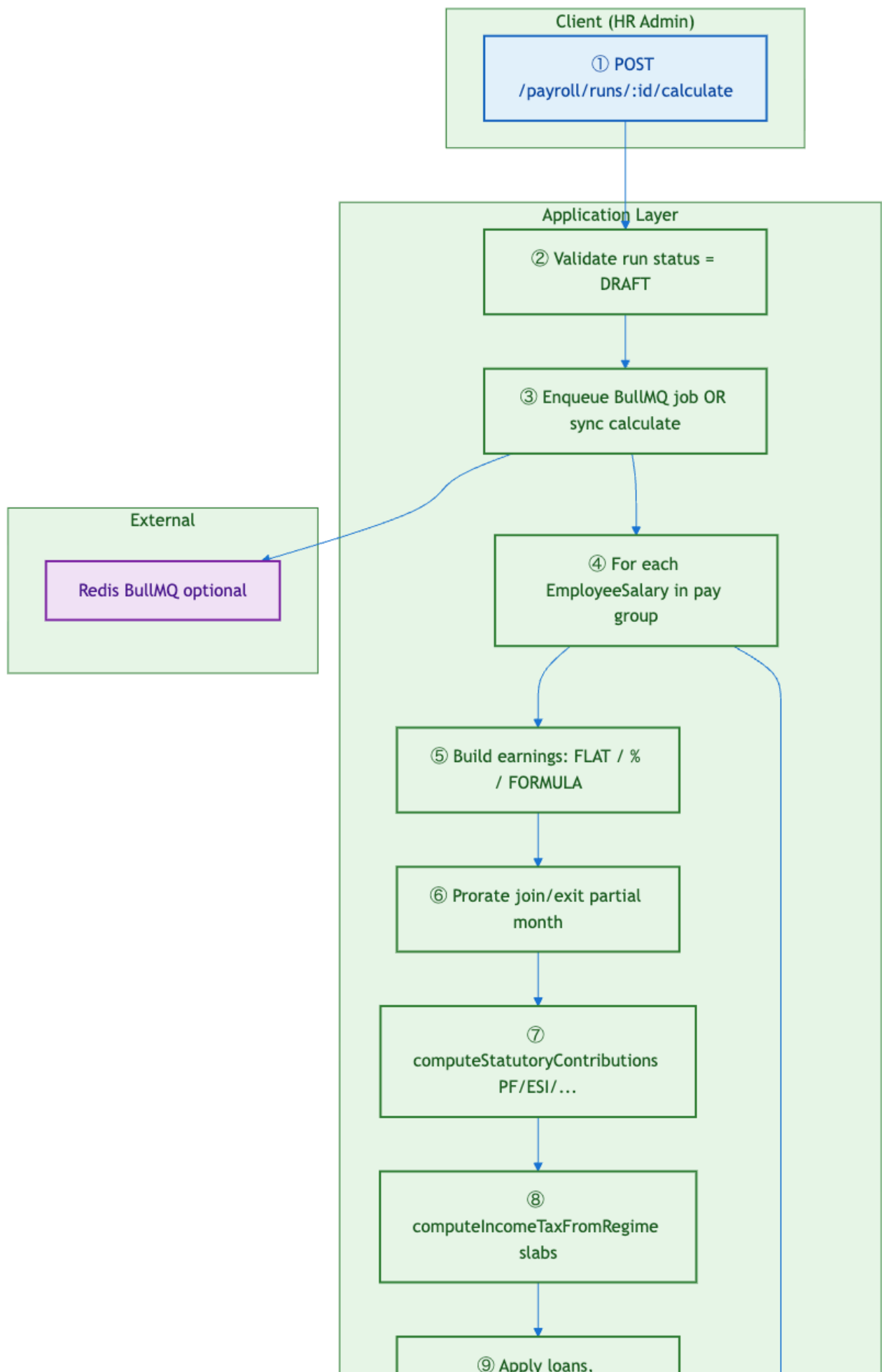
This section documents every major business flow with numbered steps and color-coded layers (blue = client, green = API, orange = database, purple = external).

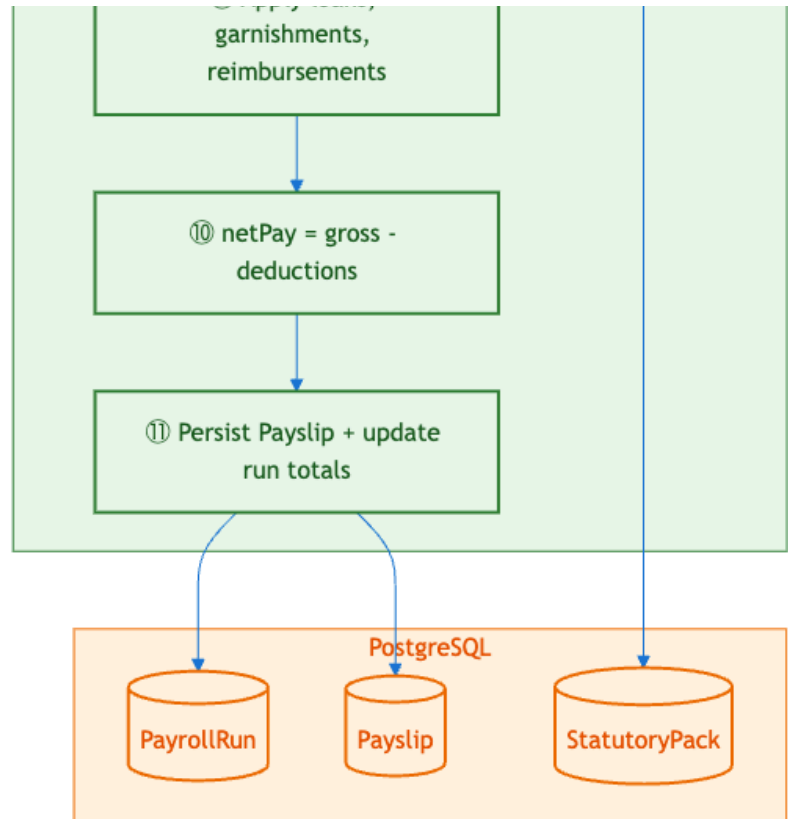
### 17.1 Payroll Engine — Gross → Deductions → Statutory → Net

**What it does:** Computes one payslip per employee for a payroll run period.

**Why it matters:** Payroll is the highest-risk domain — errors affect real money.

**How it works:** `payroll.repository.calculatePayrollRun` orchestrates components, statutory pack, tax, loans, net.



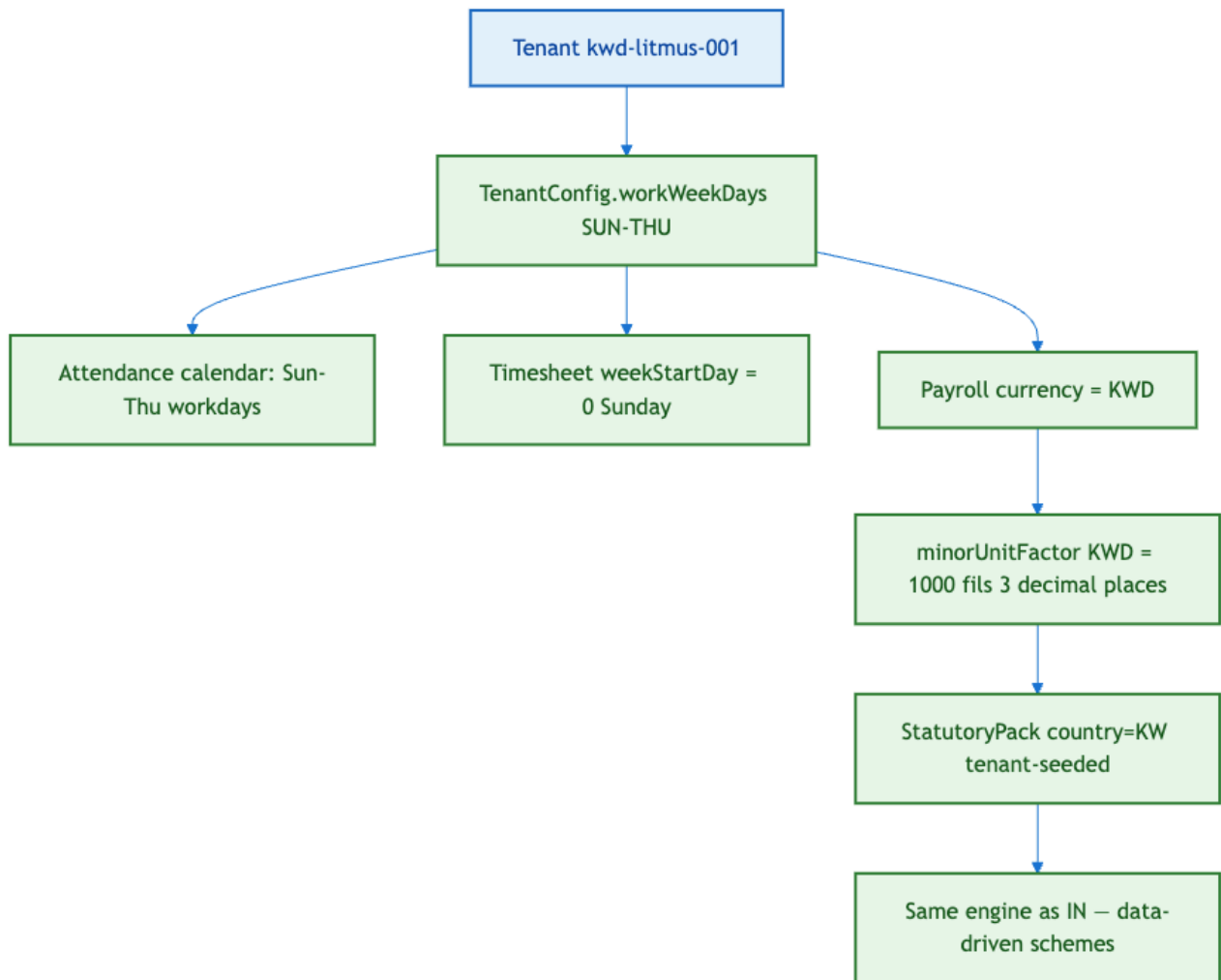


## 17.2 India Statutory — PF / ESI / PT / TDS

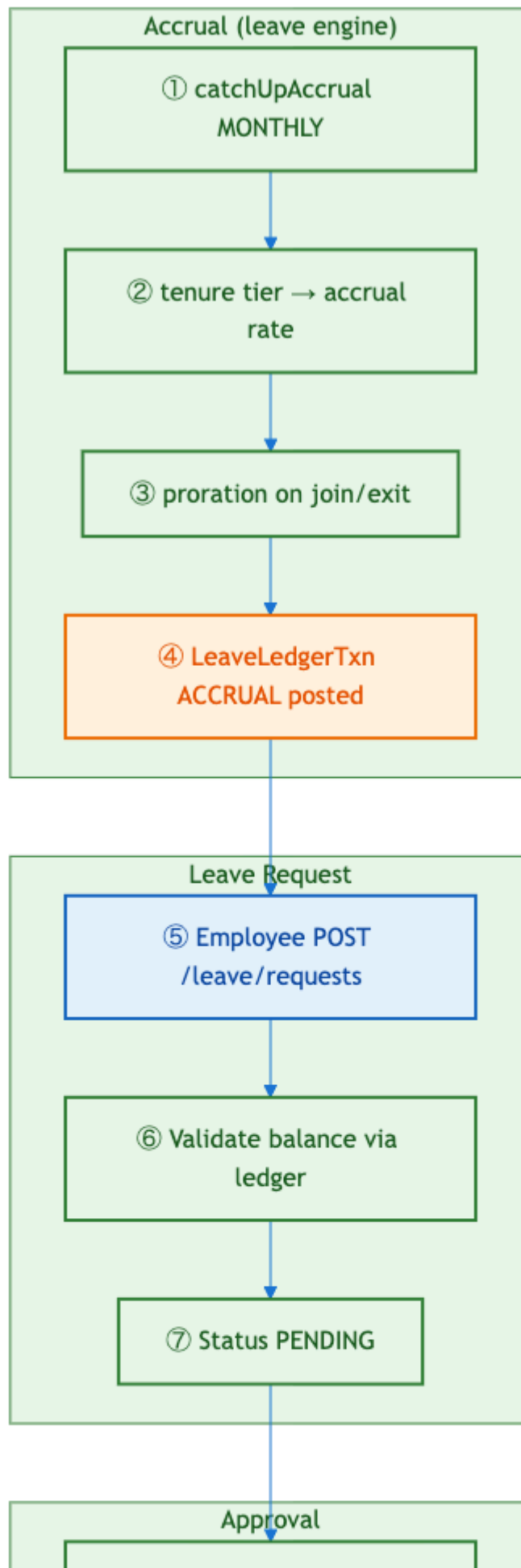


All India rules live in **StatutoryPack** JSON — not hardcoded `if (country === 'IN')` in the calculation loop.

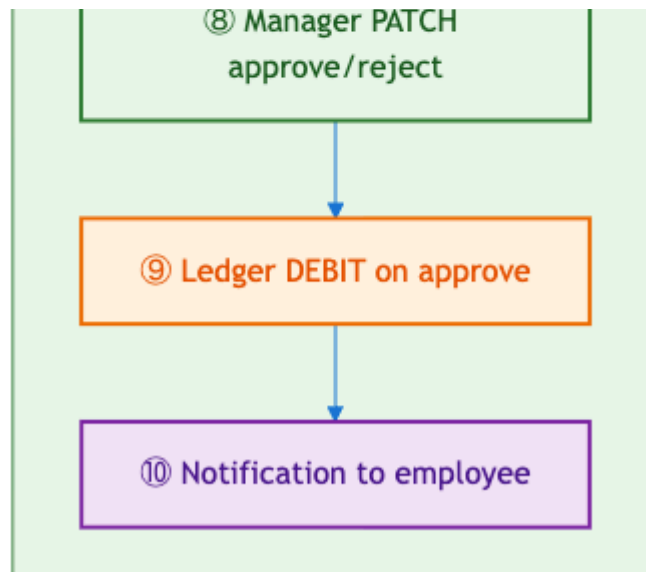
### 17.3 KWD Work Week + Currency Flow



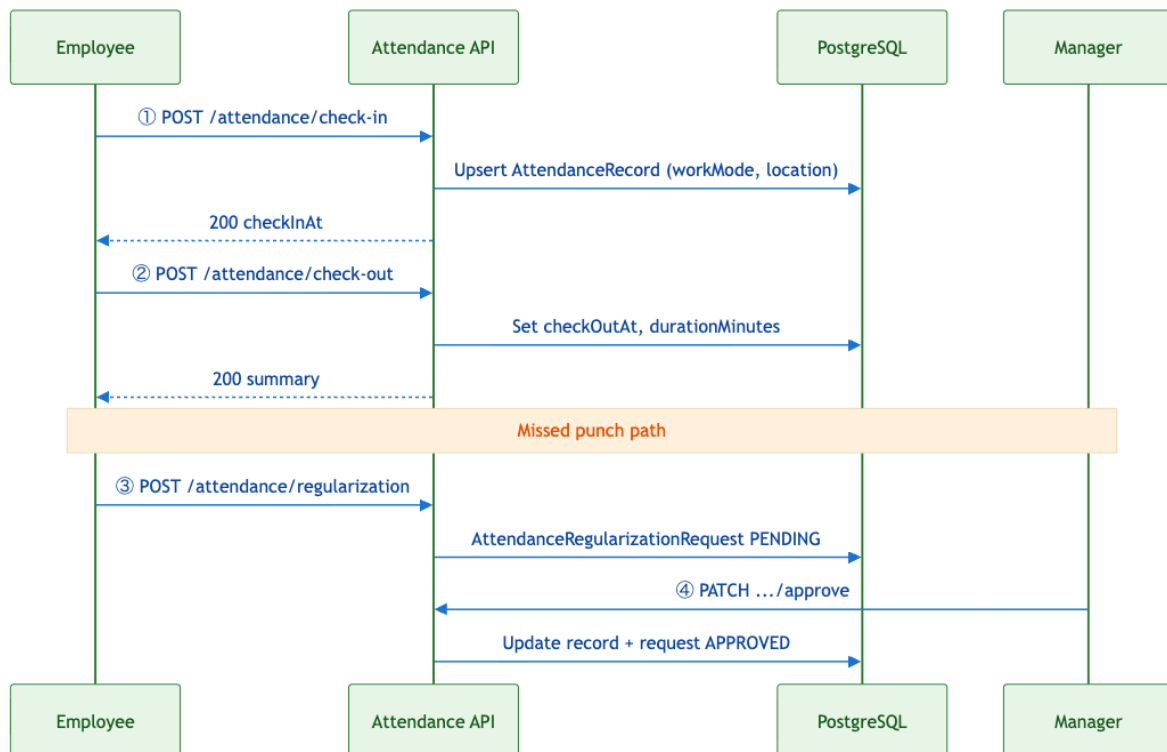
**17.4 Leave Balance Accrual + Approval**



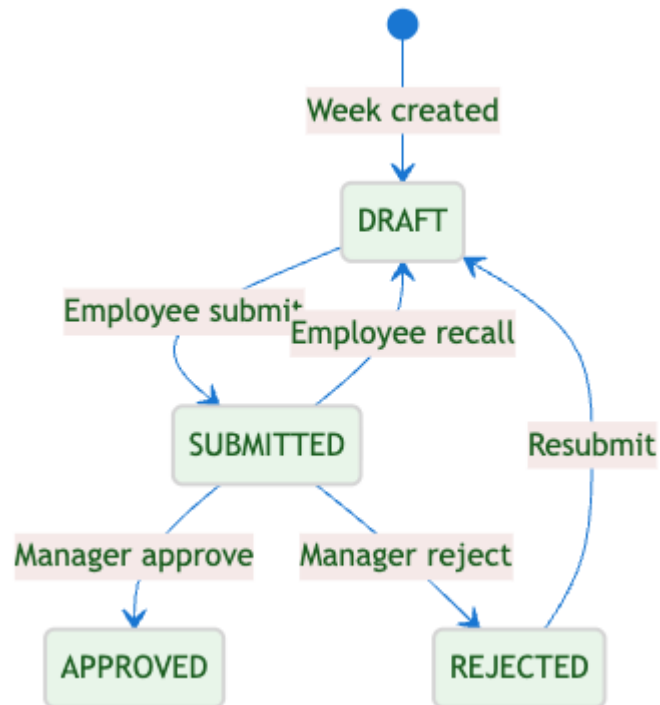




### 17.5 Attendance Check-In/Out + Regularization

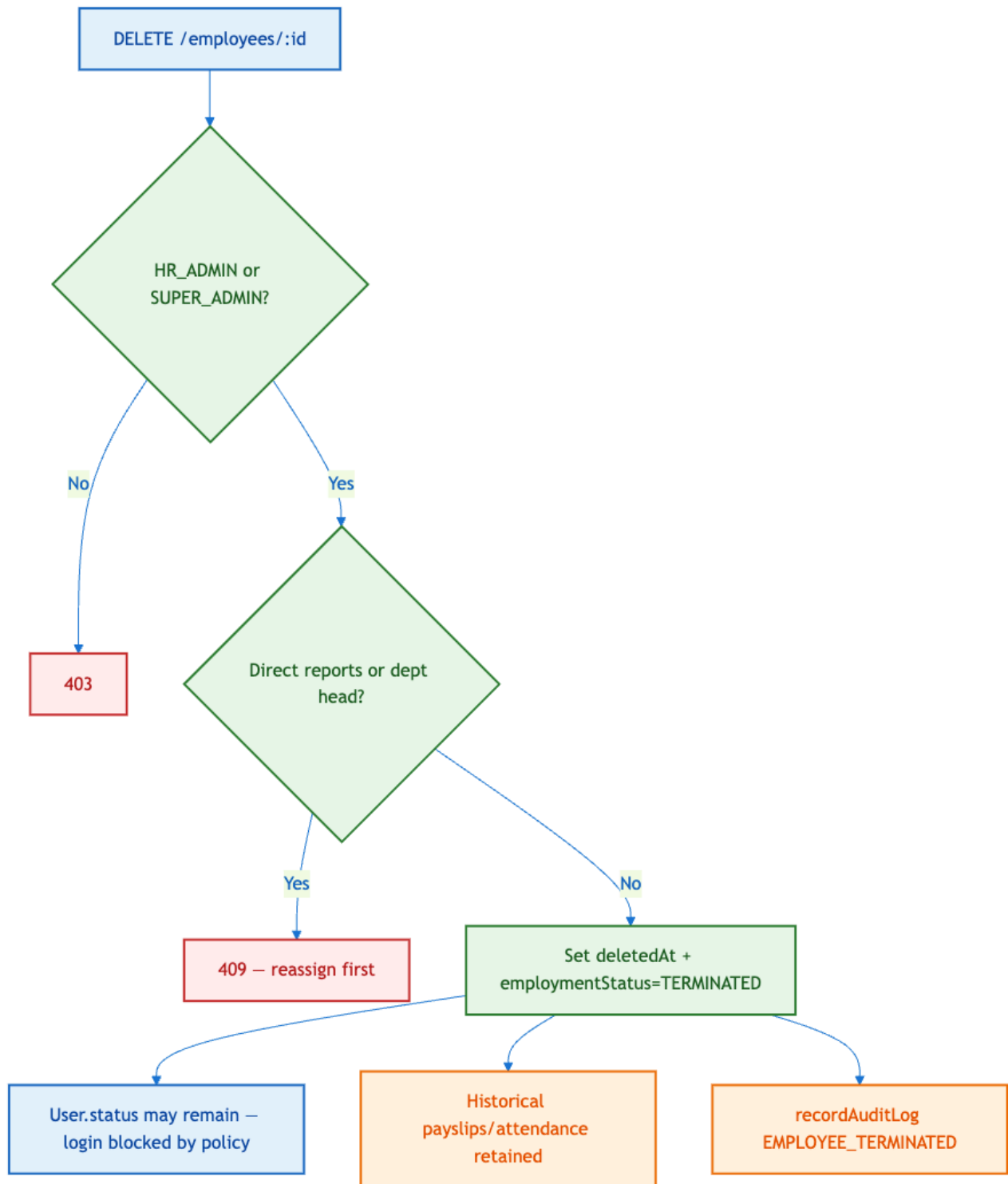


## 17.6 Timesheet Lock / Unlock / Submit

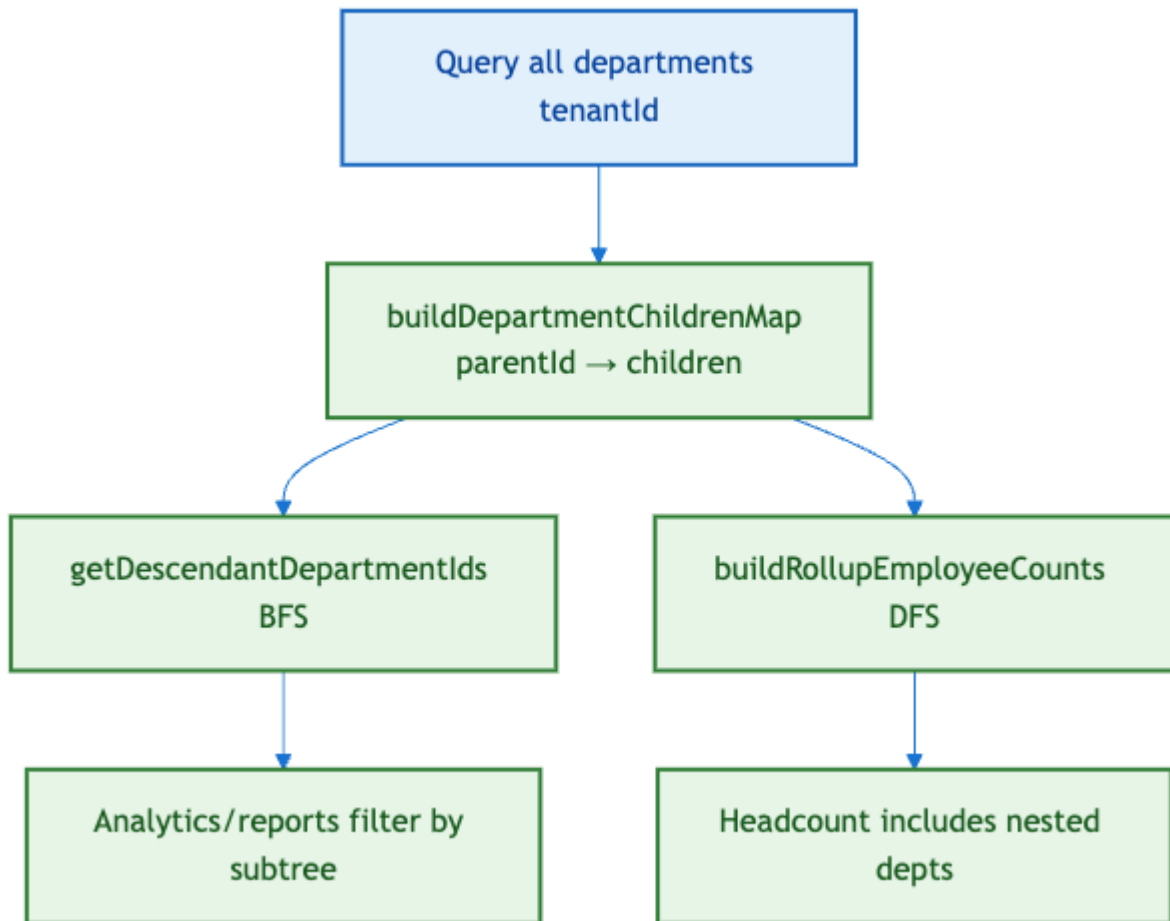


**Period locks** (HR): `POST /timesheets/locks` stores date ranges in tenant settings blob — blocks edits across those calendar dates.

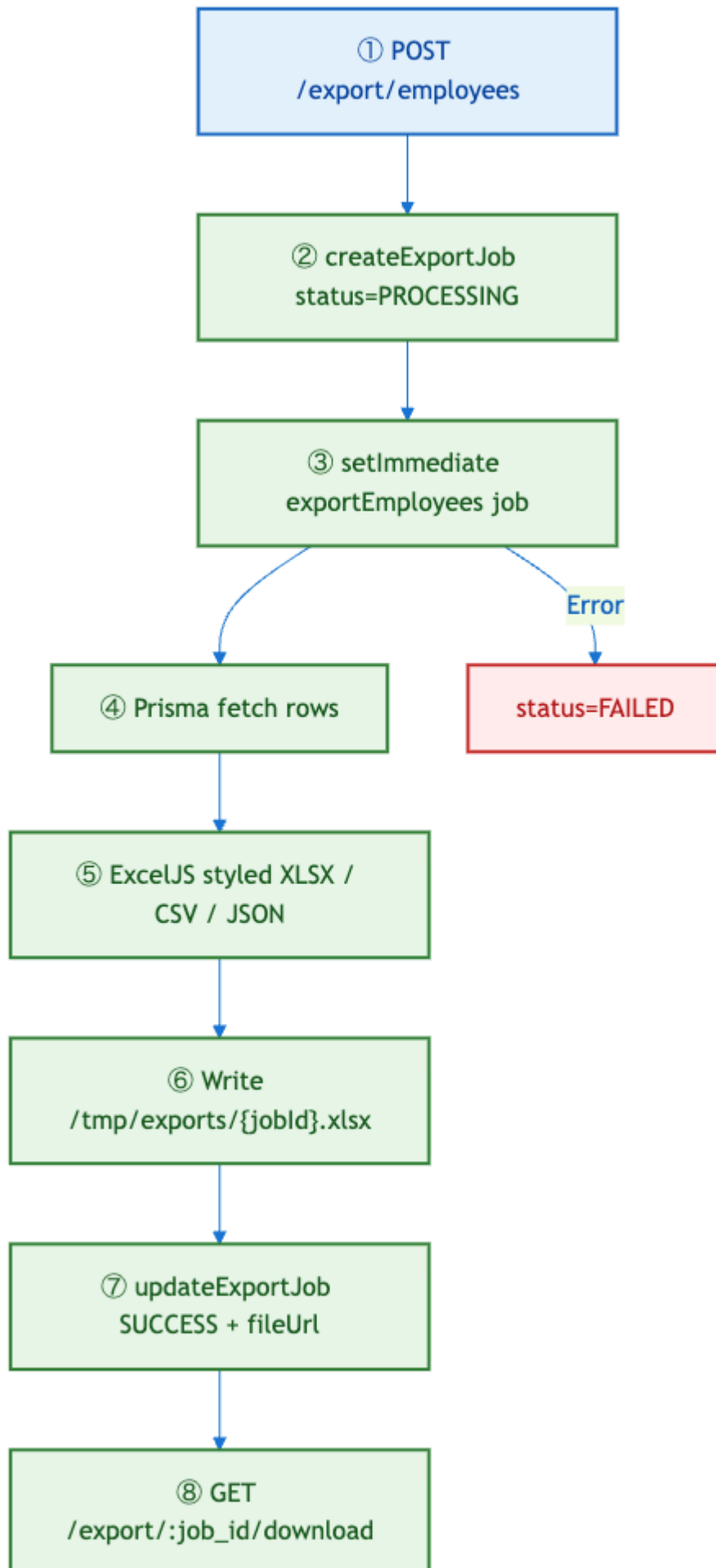
## 17.7 Employee Soft Delete Cascade



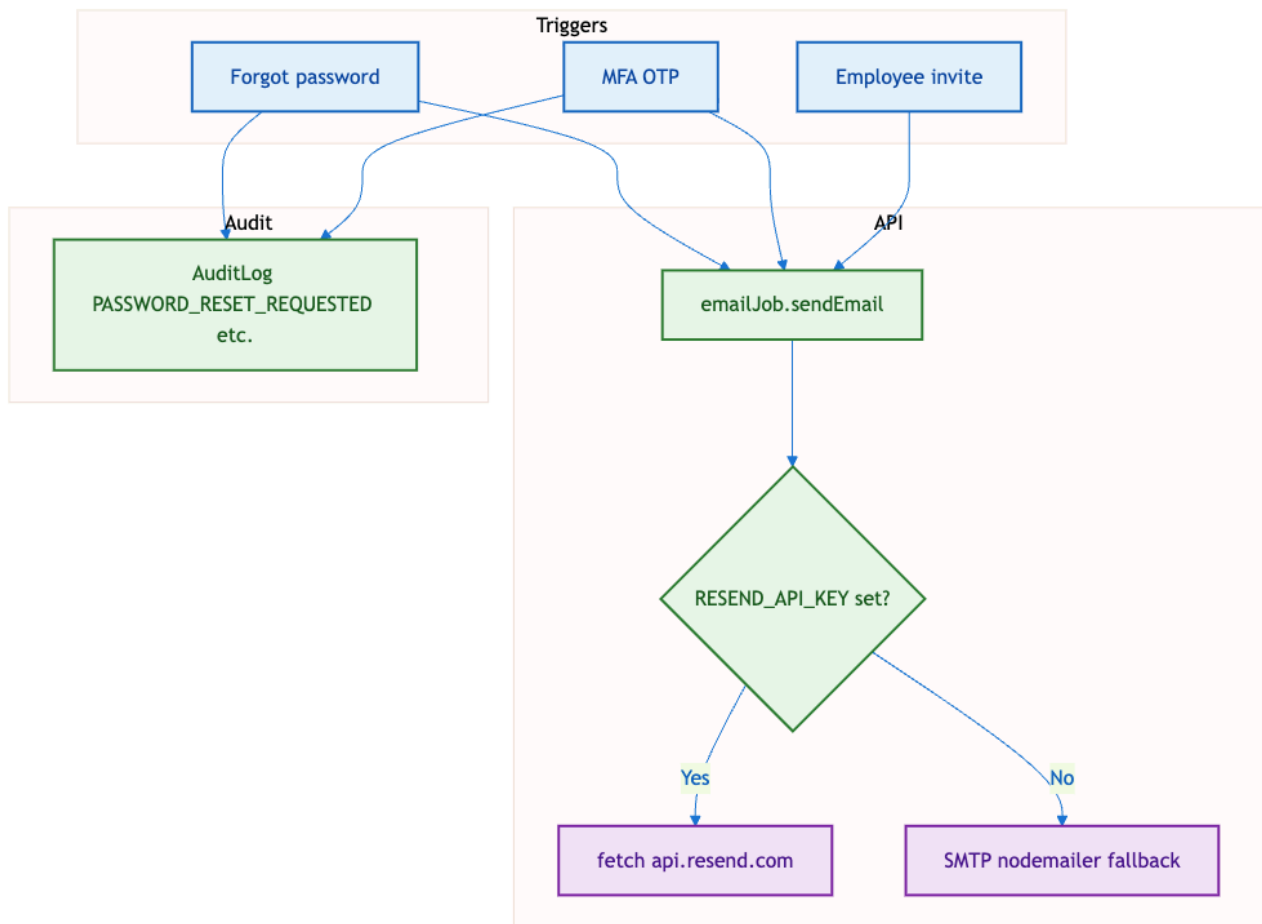
## 17.8 Department Hierarchy Tree Resolution



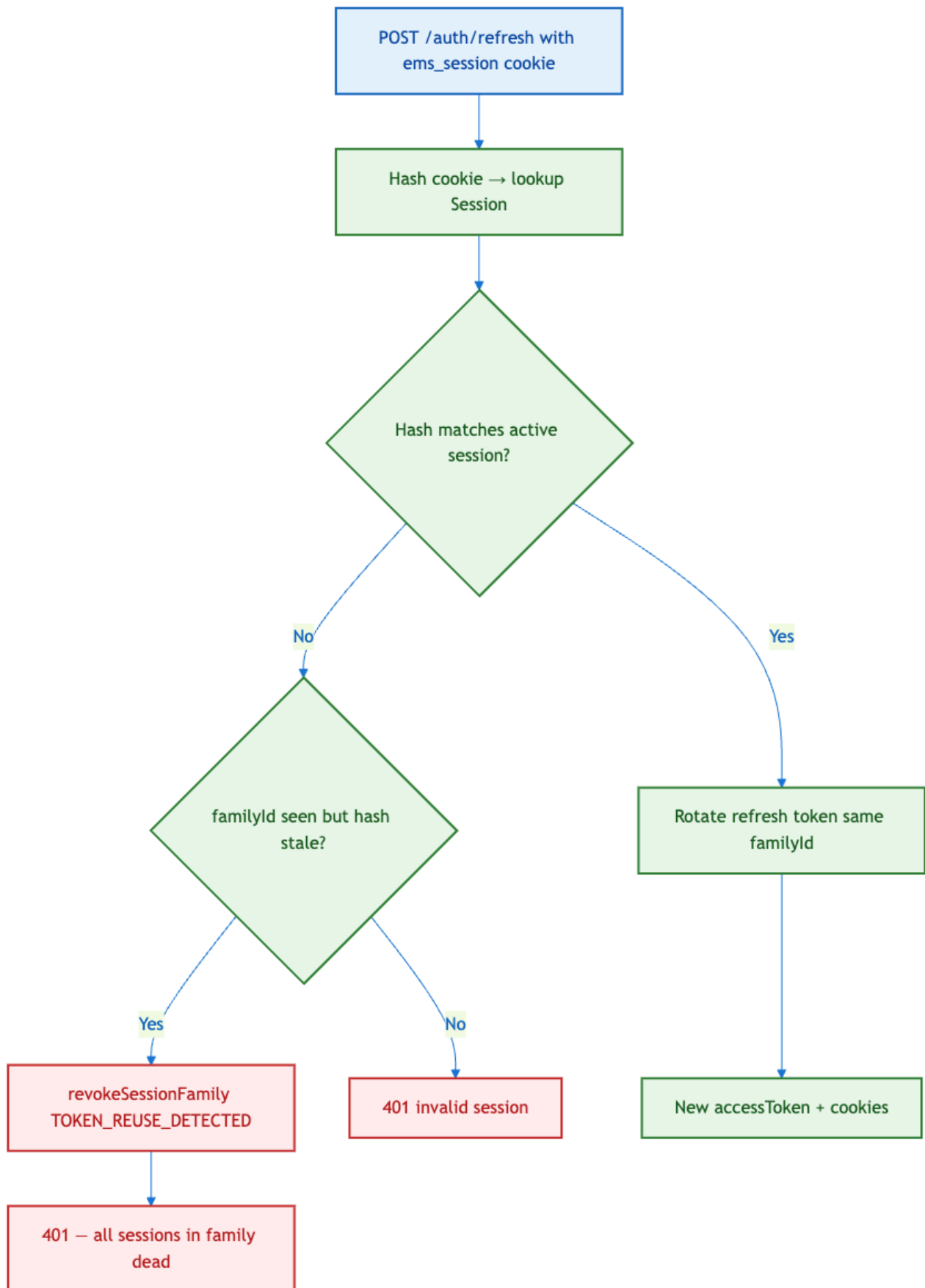
**17.9 Export Job Lifecycle (ExcelJS)**



## 17.10 Email Flow — Resend (Password Reset + OTP)

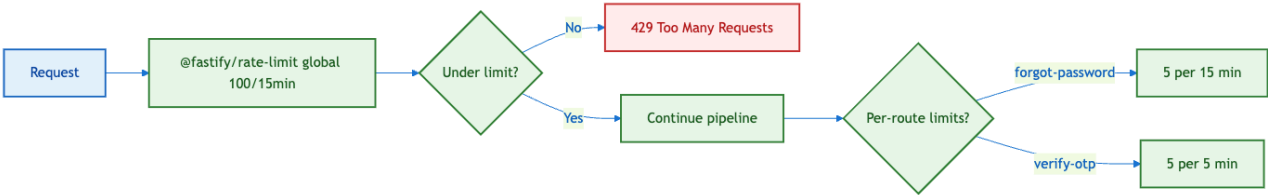


## 17.11 Session Family ID Reuse Detection

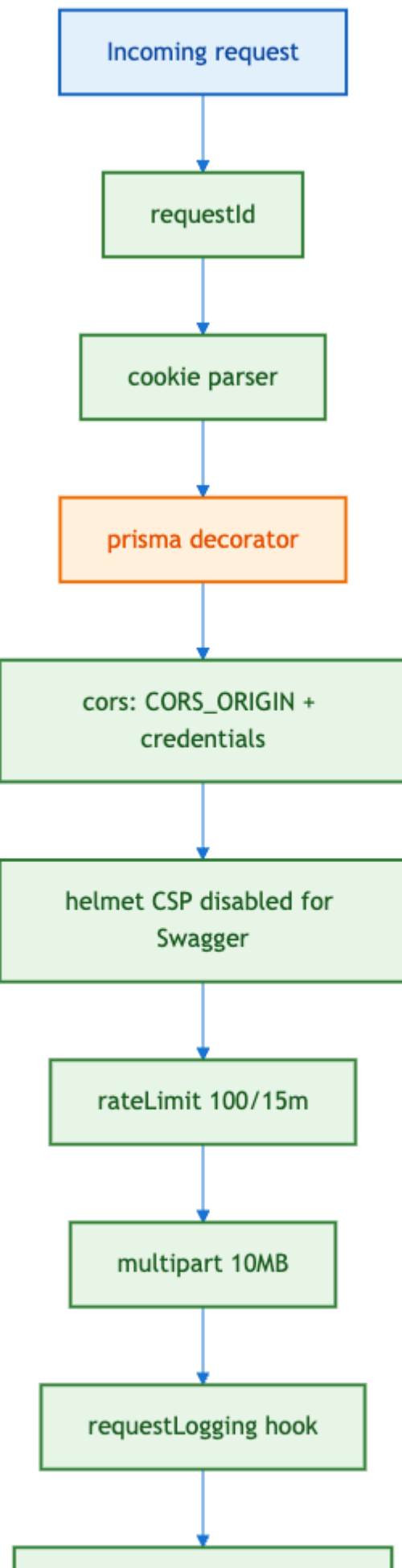




17.12 Rate Limiting Flow

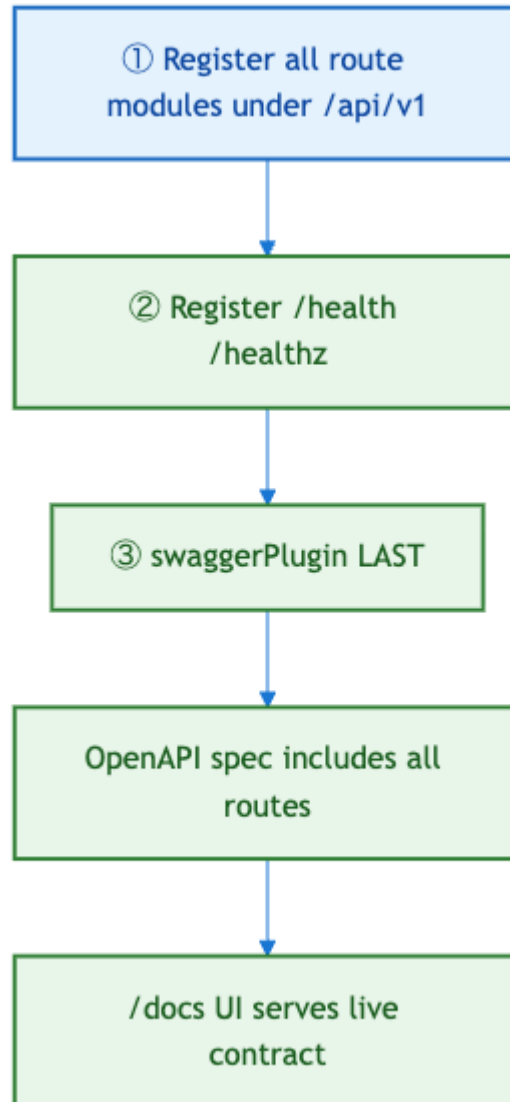


## 17.13 CORS + Helmet Middleware Chain

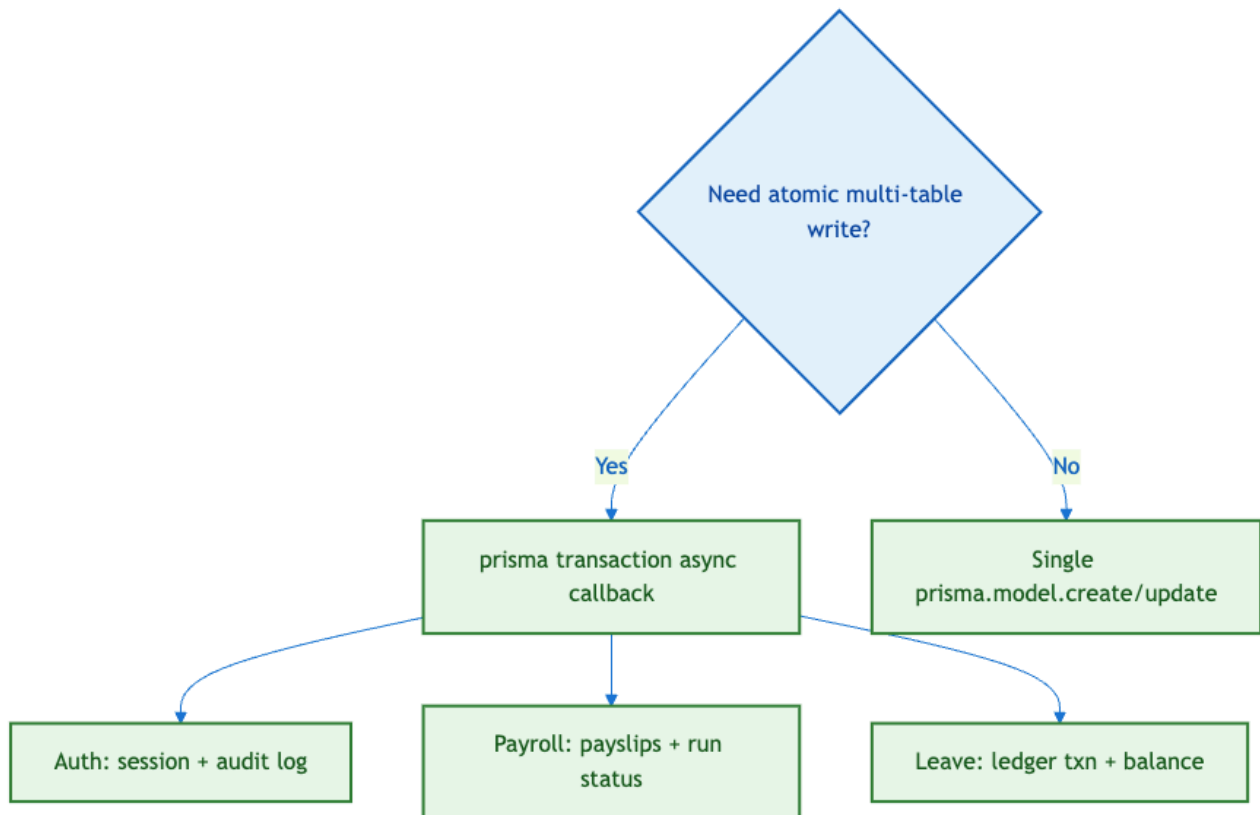


resolveTenant on /api/v1

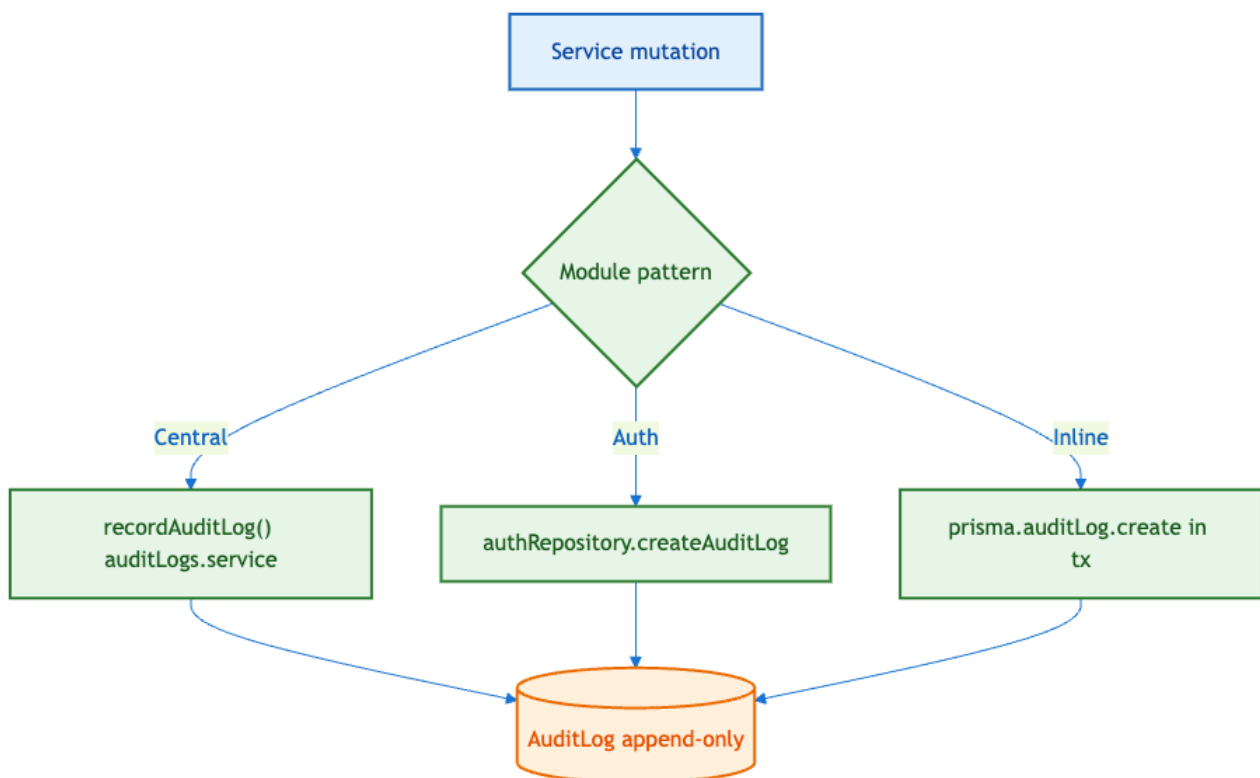
#### 17.14 Swagger Registration Order



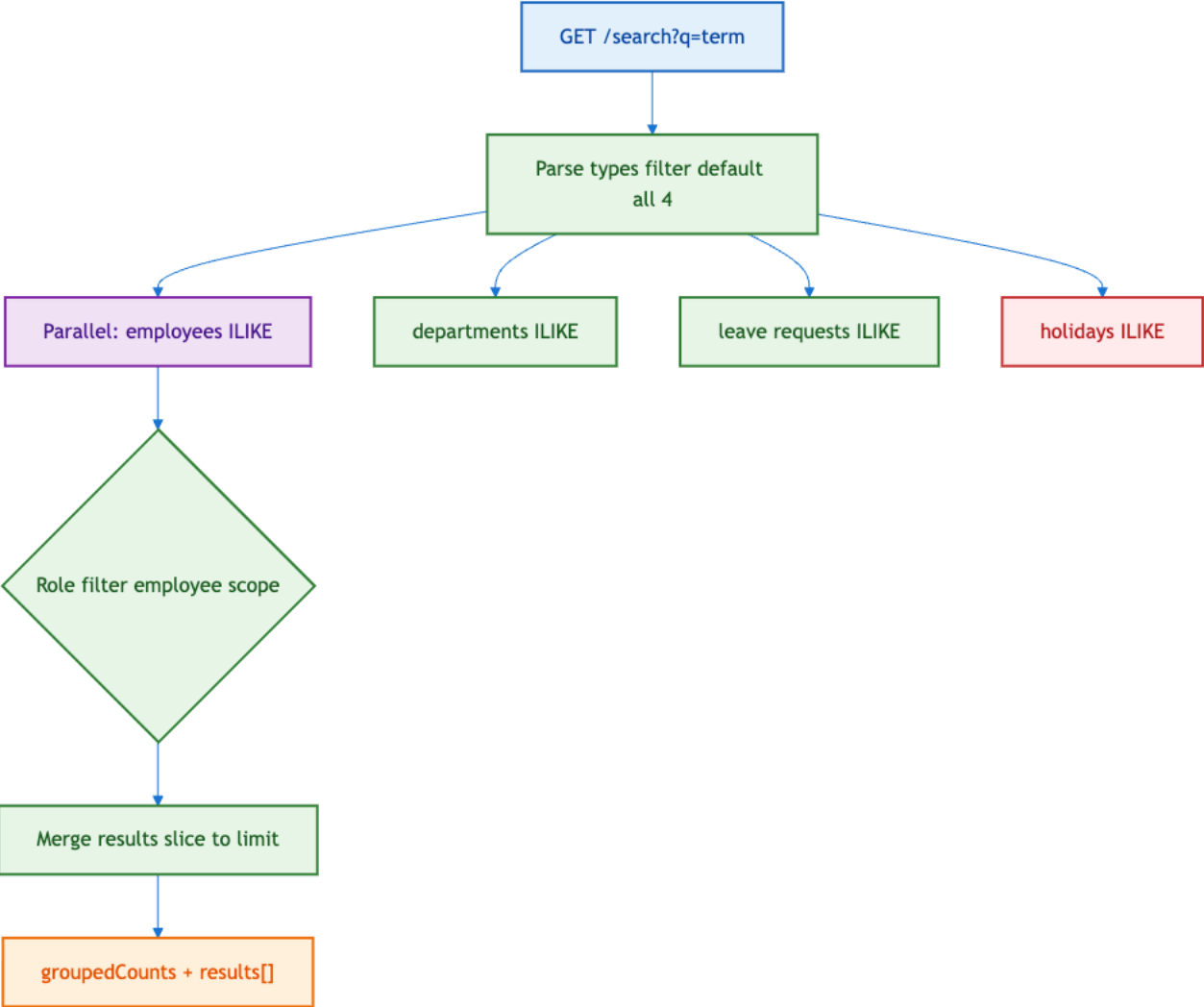
## 17.15 Prisma Transaction Patterns



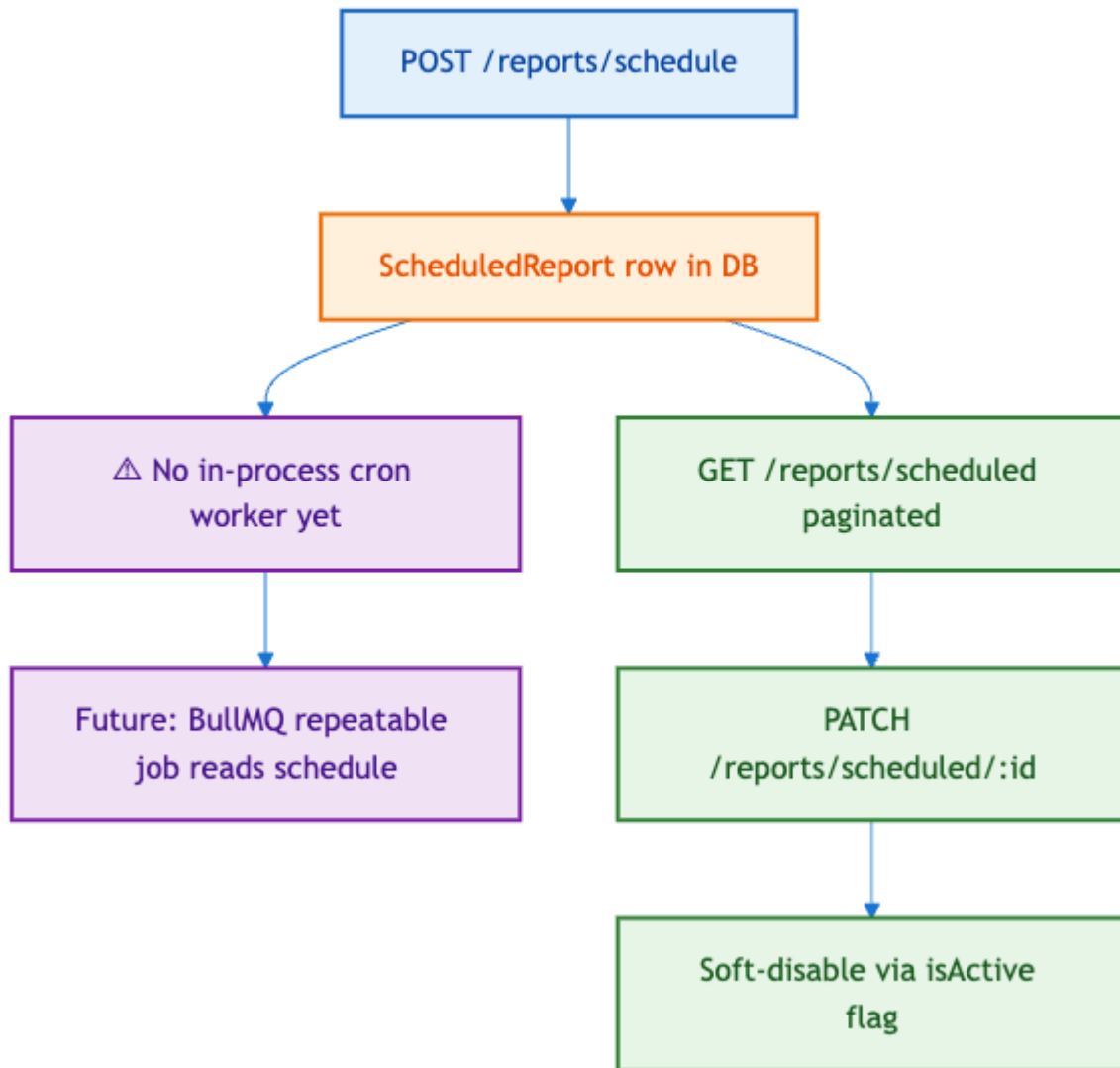
## 17.16 Audit Log Write on Mutation



17.17 Search Multi-Entity Flow



## 17.18 Report Scheduling (CRUD — cron execution)



---

---

---

---

---

---

---

---

---

---

## 18. Utilities Reference ( `src/utls/` )

| File                                  | Purpose                                                                |
|---------------------------------------|------------------------------------------------------------------------|
| <code>hash.js</code>                  | Argon2id password hash + SHA-256 for reset tokens                      |
| <code>token.js</code>                 | JWT access token creation via <code>jose</code>                        |
| <code>response.js</code>              | <code>successResponse()</code> / <code>errorResponse()</code> envelope |
| <code>id.js</code>                    | UUID v4 <code>generateId()</code>                                      |
| <code>logger.js</code>                | Shared Pino logger instance                                            |
| <code>otp.js</code>                   | OTP code generation + masking                                          |
| <code>pagination.js</code>            | Offset pagination helpers                                              |
| <code>money.js</code>                 | Currency-safe decimal formatting                                       |
| <code>statutoryCalculation.js</code>  | PF/ESI/tax slab engine                                                 |
| <code>statutoryPackShape.js</code>    | Pack DTO normalization                                                 |
| <code>payrollPeriod.js</code>         | Pay period date math                                                   |
| <code>payFrequency.js</code>          | MONTHLY/SEMI_MONTHLY cycle counts                                      |
| <code>payCalendarShape.js</code>      | Pay calendar DTO helpers                                               |
| <code>payrollComponentShape.js</code> | Salary component shapes                                                |
| <code>payrollUiShapes.js</code>       | UI-facing payroll DTOs                                                 |
| <code>formulaEval.js</code>           | Safe formula evaluation for <code>% of basis</code>                    |
| <code>workingDays.js</code>           | Working day counts with holidays                                       |
| <code>departmentTree.js</code>        | BFS descendants + rollup headcount                                     |
| <code>cloudinary.js</code>            | Upload stream + WebP pipeline                                          |
| <code>sseClients.js</code>            | In-memory SSE client map per <code>userId</code>                       |
| <code>notifier.js</code>              | Create in-app Notification rows                                        |
| <code>cycleGenerator.js</code>        | Pay cycle date generation                                              |
| <code>icsParser.js</code>             | ICS calendar import for holidays                                       |
| <code>importJobStore.js</code>        | In-memory import job status                                            |



| File     | Purpose                      |
|----------|------------------------------|
| refNo.js | Sequential reference numbers |

|  |
|--|
|  |
|  |
|  |
|  |
|  |
|  |
|  |
|  |
|  |
|  |

## 19. Middleware & Plugins Reference

**In simple terms:** Fastify hooks and plugins run in a fixed order before your route handler.  
*Changing plugin order can break auth, tenant resolution, or Swagger registration.*

### Middleware ( `src/middleware/` )

| File              | Hook                              | Responsibility                        |
|-------------------|-----------------------------------|---------------------------------------|
| resolveTenant.js  | onRequest (global under /api/v1 ) | 4-layer tenant resolution             |
| authenticate.js   | route onRequest                   | JWT verify + session revocation check |
| authenticate.js   | authorize(roles)                  | Role gate; SUPER_ADMIN bypass         |
| errorHandler.js   | setErrorHandler                   | Maps errors → JSON envelope           |
| requestLogging.js | onResponse                        | Pino request/response log line        |

### Plugins ( `src/plugins/` )

| File         | Order | Responsibility           |
|--------------|-------|--------------------------|
| requestId.js | 1     | X-Request-Id correlation |
| prisma.js    | 3     | fastify.prisma decorator |

| File                      | Order       | Responsibility                                     |
|---------------------------|-------------|----------------------------------------------------|
| <code>cors.js</code>      | 4           | Origin allowlist + credentials                     |
| <code>helmet.js</code>    | 5           | Security headers (CSP off for <code>/docs</code> ) |
| <code>rateLimit.js</code> | 6           | 100 req / 15 min global                            |
| <code>swagger.js</code>   | <b>last</b> | OpenAPI + <code>/docs</code> UI                    |
| <code>redis.js</code>     | legacy stub | Returns null — use <code>src/lib/redis.js</code>   |

**Jobs ( `src/jobs/` + `src/lib/` )**

| File                             | Role                                                            |
|----------------------------------|-----------------------------------------------------------------|
| <code>emailJob.js</code>         | Resend HTTP + SMTP fallback; password reset, OTP, invites       |
| <code>exportJob.js</code>        | ExcelJS export worker (sync <code>setImmediate</code> )         |
| <code>lib/payrollQueue.js</code> | BullMQ <code>payroll-calculate</code> queue + in-process worker |
| <code>lib/redis.js</code>        | ioredis client; cache get/set; no-op without REDIS_URL          |

|  |
|--|
|  |
|  |
|  |
|  |
|  |
|  |
|  |
|  |
|  |
|  |

**Appendix A — Glossary (Expanded)**

| Term             | Definition                                                                            |
|------------------|---------------------------------------------------------------------------------------|
| <b>tenant</b>    | One customer organization; all rows scoped by <code>tenantId</code>                   |
| <b>tenantKey</b> | Stable string for <code>X-Tenant-Key</code> header (e.g. <code>acme-corp-001</code> ) |
| <b>slug</b>      | Subdomain identifier ( <code>acme</code> from <code>acme.yourems.com</code> )         |



## Appendix B — Document Revision History

| Date      | Version    | Changes                                                                                                                     |
|-----------|------------|-----------------------------------------------------------------------------------------------------------------------------|
| July 2026 | 1.0        | Initial comprehensive technical documentation                                                                               |
| July 2026 | <b>2.0</b> | Expanded workflows, Hostinger deep dive, colored diagrams, Quick Start, full utils/middleware reference, 18+ new flowcharts |
| July 2026 | <b>3.0</b> | PDF blank-page fixes, all diagrams pre-rendered in color, section callouts, appendix reorder, HTML tables                   |

## Appendix C — Complete API Route Index

Full route list registered in `src/app.js` under `/api/v1` . See `docs/API_MAPPING.md` for request/response shapes.

### Auth & Identity

| Method | Path                           | Auth   | Roles                 |
|--------|--------------------------------|--------|-----------------------|
| POST   | <code>/auth/login</code>       | —      | any                   |
| POST   | <code>/auth/admin/login</code> | —      | HR_ADMIN, SUPER_ADMIN |
| POST   | <code>/auth/refresh</code>     | cookie | any                   |
| POST   | <code>/auth/logout</code>      | Bearer | any                   |
| POST   | <code>/auth/logout-all</code>  | Bearer | any                   |
| GET    | <code>/auth/me</code>          | Bearer | any                   |

| Method | Path                       | Auth   | Roles              |
|--------|----------------------------|--------|--------------------|
| GET    | /auth/sessions             | Bearer | any                |
| DELETE | /auth/sessions/:sessionId  | Bearer | any                |
| POST   | /auth/forgot-password      | —      | rate-limited       |
| POST   | /auth/reset-password       | —      | rate-limited       |
| GET    | /auth/validate-reset-token | —      | any                |
| POST   | /auth/verify-otp           | —      | rate-limited       |
| POST   | /auth/resend-otp           | —      | rate-limited       |
| POST   | /auth/register             | —      | creates tenant     |
| POST   | /auth/invitation/resend    | —      | public generic 200 |

## Employees & Departments

| Method                | Path                     | Notes                           |
|-----------------------|--------------------------|---------------------------------|
| GET/POST              | /employees               | CRUD; soft delete               |
| GET/PATCH/DELETE      | /employees/:id           | ?includeTerminated=true HR only |
| POST                  | /employees/:id/documents | multipart → Cloudinary WebP     |
| POST                  | /employees/:id/photo     | profile photo WebP              |
| GET                   | /employees/export/csv    | HR_ADMIN                        |
| GET/POST/PATCH/DELETE | /departments             | hierarchy tree                  |

## Attendance & Leave

| Method | Path                       | Notes              |
|--------|----------------------------|--------------------|
| POST   | /attendance/check-in       | workMode, geo      |
| POST   | /attendance/check-out      | duration calc      |
| GET    | /attendance/records        | ?month=YYYY-MM     |
| GET    | /attendance/team/records   | MANAGER+           |
| POST   | /attendance/regularization | correction request |

| Method   | Path                                   | Notes             |
|----------|----------------------------------------|-------------------|
| PATCH    | /attendance/regularization/:id/approve | MANAGER+          |
| GET/POST | /leave/requests                        | approval workflow |
| GET      | /leave/balance                         | own balances      |
| GET/POST | /leave-engine/*                        | policies, ledger  |

## Timesheets & Payroll

| Method          | Path                        | Notes                     |
|-----------------|-----------------------------|---------------------------|
| GET/POST        | /timesheets                 | weekly sheets             |
| POST            | /timesheets/:id/submit      | DRAFT→SUBMITTED           |
| PATCH           | /timesheets/:id/approve     | manager                   |
| GET/POST/DELETE | /timesheets/locks           | period locks HR           |
| GET/POST        | /payroll/runs               | payroll batches           |
| POST            | /payroll/runs/:id/calculate | BullMQ async on Hostinger |
| GET             | /payroll/runs/:id/payslips  | per-employee slips        |
| GET/POST        | /payroll/me/payout-methods  | bank details              |

## Platform & Admin

| Method    | Path              | Notes                                                                       |
|-----------|-------------------|-----------------------------------------------------------------------------|
| GET       | /analytics/*      | HR dashboards                                                               |
| GET/POST  | /reports/*        | reports + schedule CRUD                                                     |
| POST      | /export/employees | ExcelJS async job                                                           |
| GET       | /audit-logs       | immutable audit                                                             |
| GET       | /notifications    | <ul style="list-style-type: none"> <li>SSE /notifications/stream</li> </ul> |
| GET       | /search?q=        | multi-entity search                                                         |
| GET/PATCH | /settings/tenant  | tenant config                                                               |

| Method | Path                | Notes         |
|--------|---------------------|---------------|
| GET    | /manager/dashboard  | manager home  |
| GET    | /employee/dashboard | employee home |

**Health & Docs (outside prefix)**

| Method | Path       | Response      |
|--------|------------|---------------|
| GET    | /health    | {status:"ok"} |
| GET    | /healthz   | {status:"ok"} |
| GET    | /docs      | Swagger UI    |
| GET    | /docs/json | OpenAPI JSON  |

|  |
|--|
|  |
|  |
|  |
|  |
|  |
|  |
|  |
|  |
|  |
|  |

**Appendix D — Developer Troubleshooting**

| Symptom                    | Likely cause                | Fix                              |
|----------------------------|-----------------------------|----------------------------------|
| 400 MISSING_TENANT         | No tenant resolved          | Send X-Tenant-Key or login first |
| 401 TOKEN_REUSE_DETECTED   | Old refresh cookie replayed | Re-login; family revoked         |
| 403 FORBIDDEN              | Wrong memberType            | Check authorize() roles on route |
| 422 VALIDATION_ERROR       | Zod / JSON schema           | See details[] in error envelope  |
| 503 STORAGE_NOT_CONFIGURED | Missing Cloudinary env      | Set CLOUDINARY_* vars            |

| Symptom                               | Likely cause                 | Fix                                            |
|---------------------------------------|------------------------------|------------------------------------------------|
| Empty <code>{}</code> in API response | fast-json-stringify schema   | Add <code>additionalProperties: true</code>    |
| Payroll stuck CALCULATING             | Redis/BullMQ down            | Check <code>ems-redis</code> ; falls back sync |
| <code>WEEK_LOCKED</code> on timesheet | Status<br>SUBMITTED/APPROVED | Recall or manager reject first                 |

## Useful Commands

```

npm run dev           # local API with watch
npm run test:smoke    # app boots
npm run test:integration # route tests (test DB only)
npm run lint          # ESLint
npx prisma studio     # DB GUI
graphify query "auth refresh" # codebase graph search

```